```
// ===== migrations/001_identity_access.sql =====
-- ЦЕХ — этап 1 ТЗ v2: организации, пользователи, роли, изоляция данных, аудит.
--
-- Изоляция двумя слоями:
--  1) сервер проверяет право роли на действие и область (организация/площадка);
--  2) Row-Level Security: приложение работает под ролью ceh_app, и строка организации видна
--     только активному участнику этой организации — даже если код по ошибке подставит чужой
--     organization_id, база строк не вернёт.
-- Служебные операции до входа (вход по email, подтверждение, приглашение по ссылке) идут под
-- владельцем схемы в отдельном «системном» контексте сервера.

create extension if not exists citext;

do $$ begin
  if not exists (select 1 from pg_roles where rolname = 'ceh_app') then
    create role ceh_app nologin;
  end if;
end $$;

-- Контекст запроса: задаётся сервером через set_config(..., true) в начале транзакции.
create function app_user_id() returns uuid language sql stable as
$$ select nullif(current_setting('app.user_id', true), '')::uuid $$;
create function app_org_id() returns uuid language sql stable as
$$ select nullif(current_setting('app.org_id', true), '')::uuid $$;

-- ──────────────────────────── Пользователи и вход ────────────────────────────

create table users (
  id               uuid primary key default gen_random_uuid(),
  email            citext not null unique,
  full_name        text not null,
  password_hash    text not null,
  email_verified_at timestamptz,
  status           text not null default 'active' check (status in ('active', 'disabled')),
  failed_logins    integer not null default 0,
  locked_until     timestamptz,
  created_at       timestamptz not null default now(),
  updated_at       timestamptz not null default now()
);

-- Одноразовые ссылки: подтверждение email, сброс пароля. В базе только хеш токена.
create table email_tokens (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null references users(id),
  purpose     text not null check (purpose in ('verify_email', 'reset_password')),
  token_hash  text not null unique,
  expires_at  timestamptz not null,
  used_at     timestamptz,
  created_at  timestamptz not null default now()
);
```

```
create table sessions (
  id           uuid primary key default gen_random_uuid(),
  user_id      uuid not null references users(id),
  token_hash   text not null unique,
  ip           text,
  user_agent   text,
  created_at   timestamptz not null default now(),
  last_seen_at timestamptz not null default now(),
  expires_at   timestamptz not null,
  revoked_at   timestamptz
);
create index sessions_user_idx on sessions(user_id);

-- Ограничение попыток входа по адресу (ТЗ v2 §9: «ограничение попыток входа»).
create table login_attempts (
  id         bigserial primary key,
  ip         text not null,
  email      citext,
  success    boolean not null,
  created_at timestamptz not null default now()
);
create index login_attempts_ip_idx on login_attempts(ip, created_at);

-- Исходящие письма: адаптер отправки забирает их отсюда (повторная отправка без потерь).
create table outbox_emails (
  id         uuid primary key default gen_random_uuid(),
  to_email   citext not null,
  template   text not null,
  subject    text not null,
  body       text not null,
  created_at timestamptz not null default now(),
  sent_at    timestamptz,
  error      text
);

-- ─────────────────────────── Организации, площадки, участники ──────────────────────────

create table organizations (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  legal_form  text check (legal_form in ('ИП', 'ООО', 'АО', 'другое')),
  inn         text,
  phone       text,
  region      text,
  timezone    text not null default 'Europe/Moscow',
  currency    char(3) not null default 'RUB',
  -- ТЗ v1 §5: DRAFT до заполнения паспорта, CONFIGURED после проверки, COMPLIANCE_REVIEW после
  -- назначения нормативной проверки. Соответствие автоматически не выставляется.
  status      text not null default 'DRAFT' check (status in ('DRAFT', 'CONFIGURED', 'COMPLIANCE_REV
```

```
IEW', 'ARCHIVED')),
  created_by  uuid not null references users(id),
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);

create table sites (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  name            text not null,
  address         text,
  timezone        text,
  status          text not null default 'active' check (status in ('active', 'archived')),
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now()
);
create index sites_org_idx on sites(organization_id);

-- Роль — код из src/server/access/roles.ts. site_ids пусто = все площадки организации.
create table memberships (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  user_id         uuid not null references users(id),
  role            text not null,
  site_ids        uuid[] not null default '{}',
  position        text,
  status          text not null default 'active' check (status in ('active', 'blocked')),
  created_at      timestamptz not null default now(),
  blocked_at      timestamptz,
  blocked_reason  text,
  unique (organization_id, user_id)
);
create index memberships_user_idx on memberships(user_id);

create table invitations (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  email           citext not null,
  full_name       text,
  role            text not null,
  site_ids        uuid[] not null default '{}',
  position        text,
  token_hash      text not null unique,
  invited_by      uuid not null references users(id),
  expires_at      timestamptz not null,
  accepted_at     timestamptz,
  accepted_by     uuid references users(id),
  revoked_at      timestamptz,
  created_at      timestamptz not null default now()
);
```

```
create index invitations_org_idx on invitations(organization_id);

-- Активный участник организации. SECURITY DEFINER: политики других таблиц вызывают её без
-- рекурсии через политику memberships.
create function app_is_member(org uuid) returns boolean language sql stable security definer
set search_path = public as
$$ select exists (select 1 from memberships m where m.organization_id = org and m.user_id = app_user
_id() and m.status = 'active') $$;

-- ──────────────────────────── Аудит ────────────────────────────
-- T3 v1 §1.1, v2 AT-11: кто, когда, что, основание, результат. Только дописывание, цепочка
-- SHA-256 внутри организации: правка или удаление любой записи ломает проверку цепочки.

create table audit_events (
  id              bigserial primary key,
  organization_id uuid references organizations(id),
  site_id         uuid,
  actor_user_id   uuid references users(id),
  action          text not null,
  entity_type     text,
  entity_id       text,
  before          jsonb,
  after           jsonb,
  reason          text,
  result          text not null default 'success' check (result in ('success', 'denied', 'error')),
  ip              text,
  created_at      timestamptz not null default clock_timestamp(),
  prev_hash       text not null,
  hash            text not null
);
create index audit_org_idx on audit_events(organization_id, id);

create function audit_chain_payload(e audit_events) returns text language sql immutable as
$$ select concat_ws('|', e.prev_hash, coalesce(e.organization_id::text, ''), coalesce(e.site_id::tex
t, ''),
     coalesce(e.actor_user_id::text, ''), e.action, coalesce(e.entity_type, ''), coalesce(e.entity_i
d, ''),
     coalesce(e.before::text, ''), coalesce(e.after::text, ''), coalesce(e.reason, ''), e.result,
     coalesce(e.ip, ''), to_char(e.created_at at time zone 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.US')) $$;

create function audit_events_chain() returns trigger language plpgsql security definer
set search_path = public as $$
declare last text;
begin
  -- Последовательная цепочка на организацию: блокировка на время транзакции.
  perform pg_advisory_xact_lock(hashtext('audit:' || coalesce(new.organization_id::text, 'platform')
));
  select hash into last from audit_events
   where organization_id is not distinct from new.organization_id order by id desc limit 1;
  new.prev_hash := coalesce(last, 'GENESIS');
```

```
  new.created_at := clock_timestamp();
  new.hash := encode(sha256(convert_to(audit_chain_payload(new), 'UTF8')), 'hex');
  return new;
end $$;
create trigger audit_events_chain before insert on audit_events for each row execute function audit_
events_chain();

create function audit_events_immutable() returns trigger language plpgsql as $$
begin raise exception 'Записи аудита нельзя изменять или удалять'; end $$;
create trigger audit_events_no_update before update or delete on audit_events for each row execute f
unction audit_events_immutable();

-- ─────────────────────────── Row-Level Security ────────────────────────────

alter table organizations enable row level security;
create policy org_member_read on organizations for select to ceh_app using (app_is_member(id));
create policy org_member_update on organizations for update to ceh_app
  using (id = app_org_id() and app_is_member(id)) with check (id = app_org_id());

alter table sites enable row level security;
create policy sites_tenant on sites for all to ceh_app
  using (organization_id = app_org_id() and app_is_member(organization_id))
  with check (organization_id = app_org_id() and app_is_member(organization_id));

alter table memberships enable row level security;
create policy memberships_own on memberships for select to ceh_app using (user_id = app_user_id());
create policy memberships_tenant on memberships for all to ceh_app
  using (organization_id = app_org_id() and app_is_member(organization_id))
  with check (organization_id = app_org_id() and app_is_member(organization_id));

alter table invitations enable row level security;
create policy invitations_tenant on invitations for all to ceh_app
  using (organization_id = app_org_id() and app_is_member(organization_id))
  with check (organization_id = app_org_id() and app_is_member(organization_id));

alter table users enable row level security;
create policy users_self on users for select to ceh_app using (id = app_user_id());
create policy users_colleagues on users for select to ceh_app using (
  app_org_id() is not null and app_is_member(app_org_id())
  and exists (select 1 from memberships m where m.user_id = users.id and m.organization_id = app_org
_id()));
create policy users_self_update on users for update to ceh_app using (id = app_user_id()) with check
 (id = app_user_id());

alter table audit_events enable row level security;
create policy audit_tenant_read on audit_events for select to ceh_app
  using (organization_id = app_org_id() and app_is_member(organization_id));
create policy audit_insert on audit_events for insert to ceh_app
  with check ((organization_id = app_org_id() and app_is_member(organization_id))
              or (organization_id is null and actor_user_id = app_user_id()));
```

```
grant usage on schema public to ceh_app;
grant select, update (full_name, updated_at) on users to ceh_app;
grant select, update on organizations to ceh_app;
grant select, insert, update on sites, memberships, invitations to ceh_app;
grant select, insert on audit_events to ceh_app;
grant usage on sequence audit_events_id_seq to ceh_app;
-- sessions, email_tokens, login_attempts, outbox_emails — только системный контекст.

// ===== migrations/002_shift_operations.sql =====
-- ЦЕХ — этап 2 ТЗ v2: экран сотрудника (приёмка, температуры, сроки, задачи, отклонения) и дашборд
проблем.
--
-- Принципы ТЗ v2, заложенные в схему:
--  * §5: предел, частота и место контроля — только из утверждённого правила с основанием; универсал
ьной
--    температуры в коде нет. Без утверждённого правила замер сохраняется с результатом «критерий не
 задан».
--  * §5, АТ-04: замер хранит значение, единицу, объект, автора/датчик, время измерения и время внес
ения;
--    поздний ввод отмечается. Замер не изменяется и не удаляется — исправление пишется новой запись
ю.
--  * §6, АТ-02: срок годности партии — только с основанием (маркировка изготовителя и т.п.); без ос
нования
--    «Срок не подтверждён» и задача ответственному.
--  * §4, АТ-03: карантинная или незавершённая партия не попадает в доступный остаток.

-- ———————— Справочник товаров и поставщиков (минимум для приёмки; полный каталог — этап 3) ————————
——

create table products (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  name            text not null,
  unit            text not null check (unit in ('кг', 'г', 'л', 'мл', 'шт')),
  category        text,
  barcode         text,
  status          text not null default 'active' check (status in ('active', 'archived')),
  created_at      timestamptz not null default now()
);
create index products_org_idx on products(organization_id);
create unique index products_org_barcode on products(organization_id, barcode) where barcode is not
null;

create table suppliers (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  name            text not null,
  inn             text,
  status          text not null default 'active' check (status in ('active', 'archived')),
```

```
  created_at      timestamptz not null default now()
);
create index suppliers_org_idx on suppliers(organization_id);

-- ────────── Объекты контроля и правила ──────────

create table control_objects (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  site_id         uuid not null references sites(id),
  name            text not null,
  kind            text not null check (kind in ('fridge', 'freezer', 'room', 'hot_holding', 'line',
'other')),
  location        text,
  code            text not null,             -- для QR-наклейки на объекте
  status          text not null default 'active' check (status in ('active', 'archived')),
  created_at      timestamptz not null default now(),
  unique (organization_id, code)
);
create index control_objects_site_idx on control_objects(site_id);

create table control_rules (
  id                 uuid primary key default gen_random_uuid(),
  organization_id    uuid not null references organizations(id),
  control_object_id  uuid not null references control_objects(id),
  parameter          text not null default 'temperature' check (parameter in ('temperature', 'humidi
ty')),
  unit               text not null check (unit in ('°C', '%')),
  min_value          numeric,
  max_value          numeric,
  frequency_hours    numeric check (frequency_hours is null or frequency_hours > 0),
  grace_minutes      integer not null default 30 check (grace_minutes >= 0),
  source_ref         text not null,           -- основание: документ, пункт, утверждённая процедура
  reaction           text,                    -- утверждённые шаги реакции на отклонение
  allow_isolation    boolean not null default false,
  status             text not null default 'draft' check (status in ('draft', 'approved', 'retired')
),
  version            integer not null default 1,
  created_by         uuid not null references users(id),
  created_at         timestamptz not null default now(),
  approved_by        uuid references users(id),
  approved_at        timestamptz,
  retired_at         timestamptz,
  check (min_value is not null or max_value is not null),
  check (min_value is null or max_value is null or min_value <= max_value)
);
-- У объекта одновременно действует не более одного утверждённого правила на параметр.
create unique index control_rules_one_active on control_rules(control_object_id, parameter) where st
atus = 'approved';
```

```
create table measurements (
  id                 uuid primary key default gen_random_uuid(),
  organization_id    uuid not null references organizations(id),
  site_id            uuid not null references sites(id),
  control_object_id  uuid not null references control_objects(id),
  rule_id            uuid references control_rules(id),
  parameter          text not null,
  value              numeric not null,
  unit               text not null,
  measured_at        timestamptz not null,
  entered_at         timestamptz not null default now(),
  is_late            boolean not null default false,
  author_user_id     uuid references users(id),
  sensor_id          text,
  device             text,
  criterion          text,                    -- снимок критерия на момент замера
  result             text not null check (result in ('ok', 'deviation', 'no_criterion')),
  comment            text,
  corrects_id        uuid references measurements(id),
  correction_reason  text,
  check (author_user_id is not null or sensor_id is not null),
  check (corrects_id is null or correction_reason is not null)
);
create index measurements_obj_idx on measurements(control_object_id, measured_at desc);

create function measurements_immutable() returns trigger language plpgsql as $$
begin raise exception 'Замер нельзя изменить или удалить — внесите исправление новой записью с причи
ной'; end $$;
create trigger measurements_no_update before update or delete on measurements for each row execute f
unction measurements_immutable();

-- ────────── Приёмка и партии ──────────

create table receipts (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  site_id         uuid not null references sites(id),
  supplier_id     uuid references suppliers(id),
  document_no     text,
  document_date   date,
  -- §4: Черновик; Принято; На проверке; Карантин; Отказано; Возвращено.
  status          text not null default 'draft' check (status in ('draft', 'accepted', 'review', 'qu
arantine', 'rejected', 'returned')),
  comment         text,
  created_by      uuid not null references users(id),
  created_at      timestamptz not null default now(),
  completed_by    uuid references users(id),
  completed_at    timestamptz
);
create index receipts_site_idx on receipts(site_id, created_at desc);
```

```
create table lots (
  id                 uuid primary key default gen_random_uuid(),
  organization_id    uuid not null references organizations(id),
  site_id            uuid not null references sites(id),
  product_id         uuid not null references products(id),
  receipt_line_id    uuid,
  lot_no             text,
  quantity           numeric not null check (quantity >= 0),
  unit               text not null,
  received_at        timestamptz not null default now(),
  manufactured_at    date,
  expires_at         timestamptz,
  expiry_basis       text,                    -- основание срока; null — срок не подтверждён
  storage_object_id  uuid references control_objects(id),
  status             text not null check (status in ('available', 'quarantine', 'blocked', 'consumed
', 'written_off')),
  status_reason      text,
  created_at         timestamptz not null default now(),
  check (expires_at is null or expiry_basis is not null)
);
create index lots_site_idx on lots(site_id, status, expires_at);
create index lots_storage_idx on lots(storage_object_id) where status in ('available', 'quarantine')
;

create table receipt_lines (
  id                 uuid primary key default gen_random_uuid(),
  organization_id    uuid not null references organizations(id),
  receipt_id         uuid not null references receipts(id),
  product_id         uuid not null references products(id),
  quantity           numeric not null check (quantity > 0),
  unit               text not null,
  lot_no             text,
  manufactured_at    date,
  expires_at         timestamptz,
  expiry_basis       text,
  temperature        numeric,
  checks             jsonb not null default '{}',   -- packaging / labeling / documents / quality: t
rue | false
  decision           text not null check (decision in ('accepted', 'quarantine', 'rejected')),
  decision_reason    text,
  storage_object_id  uuid references control_objects(id),
  lot_id             uuid references lots(id),
  created_at         timestamptz not null default now(),
  check (expires_at is null or expiry_basis is not null),
  check (decision = 'accepted' or decision_reason is not null)
);
create index receipt_lines_receipt_idx on receipt_lines(receipt_id);
alter table lots add constraint lots_receipt_line_fk foreign key (receipt_line_id) references receip
t_lines(id);
```

```
-- ────────── Задачи, инциденты, уведомления ──────────

create table tasks (
  id               uuid primary key default gen_random_uuid(),
  organization_id  uuid not null references organizations(id),
  site_id          uuid references sites(id),
  kind             text not null,
  title            text not null,
  details          text,
  entity_type      text,
  entity_id        uuid,
  assignee_role    text,
  assignee_user_id uuid references users(id),
  due_at           timestamptz,
  status           text not null default 'open' check (status in ('open', 'done', 'cancelled')),
  result           text,
  created_by       uuid references users(id),       -- null — создано системой
  created_at       timestamptz not null default now(),
  done_by          uuid references users(id),
  done_at          timestamptz
);
create index tasks_open_idx on tasks(organization_id, status, due_at);
-- Одна открытая задача одного вида на объект: повторное событие не плодит дубли.
create unique index tasks_open_unique on tasks(organization_id, kind, entity_id) where status = 'ope
n' and entity_id is not null;

create table incidents (
  id                    uuid primary key default gen_random_uuid(),
  organization_id       uuid not null references organizations(id),
  site_id               uuid references sites(id),
  severity              text not null check (severity in ('critical', 'high', 'medium', 'low')),
  kind                  text not null check (kind in ('temperature_deviation', 'receipt_problem', 'e
xpired_lot', 'reported')),
  title                 text not null,
  what                  text not null,
  entity_type           text,
  entity_id             uuid,
  -- §2: Новая → В работе → Ожидает проверки → Закрыта.
  status                text not null default 'new' check (status in ('new', 'in_progress', 'pending
_review', 'closed')),
  responsible_role      text,
  responsible_user_id   uuid references users(id),
  due_at                timestamptz,
  rule_source           text,
  recommended_action    text,
  requires_confirmation boolean not null default true,
  action_note           text,
  action_by             uuid references users(id),
  action_at             timestamptz,
```

```
  confirmed_by          uuid references users(id),
  confirmed_at          timestamptz,
  closed_at             timestamptz,
  created_by            uuid references users(id),
  created_at            timestamptz not null default now(),
  check (status <> 'closed' or action_at is not null),
  check (status <> 'closed' or not requires_confirmation or confirmed_at is not null)
);
create index incidents_open_idx on incidents(organization_id, status, severity);

create table incident_events (
  id              bigserial primary key,
  organization_id uuid not null references organizations(id),
  incident_id     uuid not null references incidents(id),
  at              timestamptz not null default now(),
  user_id         uuid references users(id),
  event           text not null,
  note            text,
  data            jsonb
);
create index incident_events_idx on incident_events(incident_id, id);

create table notifications (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  site_id         uuid references sites(id),
  role            text,
  user_id         uuid references users(id),
  title           text not null,
  body            text,
  entity_type     text,
  entity_id       uuid,
  created_at      timestamptz not null default now(),
  check (role is not null or user_id is not null)
);
create index notifications_org_idx on notifications(organization_id, created_at desc);

create table notification_reads (
  notification_id uuid not null references notifications(id),
  user_id         uuid not null references users(id),
  organization_id uuid not null references organizations(id),
  read_at         timestamptz not null default now(),
  primary key (notification_id, user_id)
);

-- ────────── Row-Level Security: все таблицы этапа — строго внутри организации ──────────

do $$
declare t text;
begin
```

```
  foreach t in array array['products', 'suppliers', 'control_objects', 'control_rules', 'measurement
s', 'receipts', 'receipt_lines',
                           'lots', 'tasks', 'incidents', 'incident_events', 'notifications', 'notifi
cation_reads'] loop
    execute format('alter table %I enable row level security', t);
    execute format('create policy %I on %I for all to ceh_app using (organization_id = app_org_id()
and app_is_member(organization_id)) with check (organization_id = app_org_id() and app_is_member(org
anization_id))', t || '_tenant', t);
  end loop;
end $$;

grant select, insert, update on products, suppliers, control_objects, control_rules, receipts, recei
pt_lines, lots, tasks, incidents to ceh_app;
grant select, insert on measurements, incident_events, notifications, notification_reads to ceh_app;
grant delete on receipt_lines to ceh_app;  -- только строки черновика (проверка статуса — на сервере
)
grant usage on sequence incident_events_id_seq to ceh_app;

// ===== migrations/003_stock_traceability.sql =====
-- ЦЕХ — этап 3 Т3 v2: партии, склад, FEFO, карантин, списания, прослеживаемость; единый каталог MON
 CHEF.
--
--  * Справочник платформы (reference_products) — общий для всех организаций, загружается из единого
 каталога
--    MON CHEF: каждая исходная строка — отдельный ProductID, дубли не сливаются автоматически (Т3 v
1 AT-05),
--    пищевая ценность хранится со статусом проверки (REVIEW — не подтверждено источником).
--  * Конвертация единиц — только по подтверждённому коэффициенту (Т3 v1 §6, AT-06).
--  * Движение запаса — неизменяемый журнал stock_movements; остаток партии меняется только движение
м.
--  * Повтор операции с тем же operation_id не создаёт второго движения (Т3 v2 AT-12).

create table reference_products (
  id               uuid primary key default gen_random_uuid(),
  code             text not null unique,            -- ProductID MON CHEF (PROD-0001)
  source           text not null,
  name             text not null,
  product_type     text,
  basis            text,
  kcal_100         numeric, protein_100 numeric, fat_100 numeric, carbs_100 numeric, fiber_100 numer
ic, sugars_100 numeric,
  storage          text, shelf_room text, shelf_fridge text, shelf_freezer text, spoilage text,
  verification_status text not null default 'REVIEW' check (verification_status in ('REVIEW', 'VERIF
IED', 'REJECTED')),
  review_flags     text,
  source_file      text, source_row integer, original_id text,
  duplicate_group  text,                            -- группа одинаковых названий: «не объединять ав
томатически»
  imported_at      timestamptz not null default now()
);
```

```
create index reference_products_name_idx on reference_products (lower(name));
grant select on reference_products to ceh_app;

alter table products add column reference_product_id uuid references reference_products(id);

create table unit_conversions (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  product_id      uuid not null references products(id),
  from_unit       text not null check (from_unit in ('кг', 'г', 'л', 'мл', 'шт')),
  to_unit         text not null check (to_unit in ('кг', 'г', 'л', 'мл', 'шт')),
  factor          numeric not null check (factor > 0),     -- 1 from_unit = factor to_unit
  source          text not null,                           -- основание: взвешивание, спецификация,
маркировка
  confirmed_by    uuid not null references users(id),
  created_at      timestamptz not null default now(),
  unique (product_id, from_unit, to_unit),
  check (from_unit <> to_unit)
);

-- Идемпотентность: результат операции хранится по ключу; повтор возвращает тот же результат.
create table operations (
  organization_id uuid not null references organizations(id),
  operation_id    text not null check (length(operation_id) between 8 and 100),
  kind            text not null,
  result          jsonb not null,
  created_by      uuid references users(id),
  created_at      timestamptz not null default now(),
  primary key (organization_id, operation_id)
);

create table stock_movements (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  site_id         uuid not null references sites(id),
  lot_id          uuid not null references lots(id),
  product_id      uuid not null references products(id),
  kind            text not null check (kind in ('receipt', 'consume', 'write_off', 'transfer_out', '
transfer_in', 'count_adjust', 'return_to_supplier')),
  quantity        numeric not null check (quantity <> 0),   -- приход «+», расход «−», в единице пар
тии
  unit            text not null,
  reason_code     text,
  reason          text,
  ref_type        text,
  ref_id          uuid,
  operation_id    text,
  fefo_override   boolean not null default false,
  created_by      uuid references users(id),
  created_at      timestamptz not null default now()
```

```
);
create index stock_movements_lot_idx on stock_movements(lot_id, created_at);
create index stock_movements_ref_idx on stock_movements(ref_type, ref_id);

create function stock_movements_immutable() returns trigger language plpgsql as $$
begin raise exception 'Движение запаса нельзя изменить или удалить — проведите корректирующее движен
ие'; end $$;
create trigger stock_movements_no_update before update or delete on stock_movements for each row exe
cute function stock_movements_immutable();

-- Связи партий: перемещение на другую площадку (и дальше — производство) создаёт дочернюю партию.
create table lot_links (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  parent_lot_id   uuid not null references lots(id),
  child_lot_id    uuid not null references lots(id),
  kind            text not null check (kind in ('transfer', 'production')),
  quantity        numeric not null check (quantity > 0),
  unit            text not null,
  created_at      timestamptz not null default now()
);
create index lot_links_parent_idx on lot_links(parent_lot_id);
create index lot_links_child_idx on lot_links(child_lot_id);

create table stock_counts (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  site_id         uuid not null references sites(id),
  status          text not null default 'open' check (status in ('open', 'closed', 'cancelled')),
  comment         text,
  created_by      uuid not null references users(id),
  created_at      timestamptz not null default now(),
  closed_by       uuid references users(id),
  closed_at       timestamptz
);
create unique index stock_counts_one_open on stock_counts(site_id) where status = 'open';

create table stock_count_lines (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  count_id        uuid not null references stock_counts(id),
  lot_id          uuid not null references lots(id),
  expected        numeric not null,
  counted         numeric check (counted is null or counted >= 0),
  note            text,
  counted_by      uuid references users(id),
  counted_at      timestamptz,
  unique (count_id, lot_id)
);
```

```
do $$
declare t text;
begin
  foreach t in array array['unit_conversions', 'operations', 'stock_movements', 'lot_links', 'stock_
counts', 'stock_count_lines'] loop
    execute format('alter table %I enable row level security', t);
    execute format('create policy %I on %I for all to ceh_app using (organization_id = app_org_id()
and app_is_member(organization_id)) with check (organization_id = app_org_id() and app_is_member(org
anization_id))', t || '_tenant', t);
  end loop;
end $$;

grant select, insert, update on unit_conversions, stock_counts, stock_count_lines to ceh_app;
grant select, insert on operations, stock_movements, lot_links to ceh_app;
grant update (reference_product_id) on products to ceh_app;

// ===== migrations/004_regulatory_registry.sql =====
-- ЦЕХ — этап 4 Т3 v2: реестр нормативных требований, применимость, версии, даты, проверка специалис
том.
--
-- Т3 v2 §7: для каждого требования — источник, вид/номер/издатель, официальный URL/файл, пункт, юри
сдикция,
-- обязательность, область применения, процессы и продукты, даты публикации/начала/окончания действи
я,
-- переходные положения, редакция, даты последней и следующей проверки, проверивший, статус проверки
,
-- доказательство и владелец актуализации. AI или сторонняя статья не могут сами присвоить «Действуе
т».
-- AT-08: без даты начала действия, области применения и проверки специалистом редакция не активируе
тся.
-- AT-09: новая редакция сохраняет историю, сравнивается с прежней и открывает задачи пересмотра про
цедур.

-- Нормативный профиль предприятия: по нему система предлагает применимость, решает специалист.
create table org_regulatory_profiles (
  organization_id uuid primary key references organizations(id),
  object_types    text[] not null default '{}',   -- restaurant, cafe, canteen, catering, delivery,
school, …
  processes       text[] not null default '{}',   -- receiving, storage, cold_prep, thermal, cooling
, serving, delivery, cleaning, staff, packaging
  product_groups  text[] not null default '{}',
  sales_modes     text[] not null default '{}',   -- hall, takeaway, delivery, catering
  updated_by      uuid references users(id),
  updated_at      timestamptz not null default now()
);

create table reg_sources (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  code            text not null,                  -- краткое обозначение: СанПиН 2.3/2.4.4282-26
```

```
  kind            text not null check (kind in ('СанПиН', 'ТР ТС', 'ФЗ', 'ГОСТ', 'МУК', 'Постановлен
ие', 'Приказ', 'Внутренний', 'Другое')),
  title           text not null,                  -- официальное название
  issuer          text,
  official_url    text,
  jurisdiction    text not null default 'РФ' check (jurisdiction in ('РФ', 'ЕАЭС', 'Субъект РФ', 'Ор
ганизация')),
  binding         text not null default 'unknown' check (binding in ('mandatory', 'voluntary', 'meth
odological', 'unknown')),
  note            text,
  created_by      uuid references users(id),
  created_at      timestamptz not null default now(),
  unique (organization_id, code)
);

create table reg_source_versions (
  id                    uuid primary key default gen_random_uuid(),
  organization_id       uuid not null references organizations(id),
  source_id             uuid not null references reg_sources(id),
  edition               text not null,            -- редакция: «ред. от …», «первоначальная»
  published_at          date,
  effective_from        date,
  effective_to          date,
  transitional          text,                     -- переходные положения
  file_ref              text,                     -- официальный файл/URL редакции
  -- Порядок работы: Проект → Требует проверки → Проверен специалистом → Утверждён для применения.
  -- «Скоро вступает в силу» / «Действует» / «Утратил силу» вычисляются из дат утверждённой редакции
.
  workflow_status       text not null default 'needs_review'
                        check (workflow_status in ('draft', 'needs_review', 'checked', 'approved', '
suspended', 'archived')),
  check_evidence        text,                     -- где и как сверено (официальный текст, номер пу
бликации)
  last_checked_at       timestamptz,
  checked_by            uuid references users(id),
  next_check_at         date,
  approved_by           uuid references users(id),
  approved_at           timestamptz,
  owner_user_id         uuid references users(id), -- владелец актуализации
  supersedes_version_id uuid references reg_source_versions(id),
  data_origin           text,                     -- откуда данные: «ТЗ v2 §8 — требует сверки», вв
од специалиста
  created_by            uuid references users(id),
  created_at            timestamptz not null default now(),
  check (effective_to is null or effective_from is null or effective_to > effective_from),
  check (workflow_status not in ('checked', 'approved') or (last_checked_at is not null and checked_
by is not null and check_evidence is not null)),
  check (workflow_status <> 'approved' or (effective_from is not null and approved_by is not null an
d owner_user_id is not null and next_check_at is not null))
);
```

```
create index reg_versions_source_idx on reg_source_versions(source_id, created_at);

create table reg_requirements (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  version_id      uuid not null references reg_source_versions(id),
  clause          text not null,                  -- пункт: «п. 2.5», «ст. 10 ч. 3»
  text            text not null,                  -- формулировка из официального текста
  scope           text not null,                  -- область применения
  object_types    text[] not null default '{}',
  processes       text[] not null default '{}',
  product_groups  text[] not null default '{}',
  created_by      uuid references users(id),
  created_at      timestamptz not null default now(),
  unique (version_id, clause)
);

-- Решение о применимости требования к предприятию (нормативный профиль). Предложение системы ≠ реше
ние.
create table reg_applicability (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  requirement_id  uuid not null references reg_requirements(id),
  applicable      boolean not null,
  rationale       text not null,
  decided_by      uuid not null references users(id),
  decided_at      timestamptz not null default now(),
  unique (requirement_id)
);

-- Связь требование → контрольная процедура (правило контроля) → журнал/запись.
alter table control_rules add column requirement_id uuid references reg_requirements(id);

-- Пересмотр при новой редакции: сравнение пунктов и задачи на пересмотр связанных процедур.
create table reg_reviews (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  old_version_id  uuid not null references reg_source_versions(id),
  new_version_id  uuid not null references reg_source_versions(id),
  diff            jsonb not null,                 -- { added: [...], removed: [...], changed: [...]
 }
  affected_rules  integer not null default 0,
  created_by      uuid references users(id),
  created_at      timestamptz not null default now()
);

do $$
declare t text;
begin
  foreach t in array array['org_regulatory_profiles', 'reg_sources', 'reg_source_versions', 'reg_req
```

```
uirements', 'reg_applicability', 'reg_reviews'] loop
    execute format('alter table %I enable row level security', t);
    execute format('create policy %I on %I for all to ceh_app using (organization_id = app_org_id()
and app_is_member(organization_id)) with check (organization_id = app_org_id() and app_is_member(org
anization_id))', t || '_tenant', t);
  end loop;
end $$;

grant select, insert, update on org_regulatory_profiles, reg_sources, reg_source_versions, reg_requi
rements, reg_applicability to ceh_app;
grant select, insert on reg_reviews to ceh_app;

// ===== migrations/005_journals.sql =====
-- ЦЕХ — этап 5 ТЗ v2 (часть 1): журналы ХАССП из событий.
--
-- Принцип ТЗ v2 §0: сотрудник выполняет операцию один раз — журналы J01–J53 формируются из событий
системы
-- (замеры, приёмка, партии, отклонения). Для форм, у которых ещё нет событий, — ручная запись по ст
олбцам
-- утверждённого шаблона; запись неизменяема, исправление — новой записью с причиной (ТЗ v2 §10, AT-
11).
-- Подпись журнала за период хранит хеш содержимого: правка после подписи видна.

create table journal_entries (
  id                uuid primary key default gen_random_uuid(),
  organization_id   uuid not null references organizations(id),
  site_id           uuid not null references sites(id),
  form_code         text not null check (form_code ~ '^J\d{2}$'),
  occurred_at       timestamptz not null,              -- когда выполнено (факт)
  entered_at        timestamptz not null default now(),-- когда внесено
  is_late           boolean not null default false,
  values            jsonb not null,                    -- значения по столбцам шаблона
  author_user_id    uuid not null references users(id),
  corrects_id       uuid references journal_entries(id),
  correction_reason text,
  check (corrects_id is null or correction_reason is not null)
);
create index journal_entries_form_idx on journal_entries(organization_id, site_id, form_code, occurr
ed_at);

create function journal_entries_immutable() returns trigger language plpgsql as $$
begin raise exception 'Запись журнала нельзя изменить или удалить — внесите исправление с причиной';
 end $$;
create trigger journal_entries_no_update before update or delete on journal_entries for each row exe
cute function journal_entries_immutable();

create table journal_signoffs (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  site_id         uuid not null references sites(id),
```

```
  form_code       text not null,
  period_from     date not null,
  period_to       date not null,
  rows_count      integer not null,
  content_hash    text not null,
  note            text,
  signed_by       uuid not null references users(id),
  signed_at       timestamptz not null default now(),
  check (period_to >= period_from)
);
create index journal_signoffs_idx on journal_signoffs(organization_id, site_id, form_code, period_fr
om);

create function journal_signoffs_immutable() returns trigger language plpgsql as $$
begin raise exception 'Подпись журнала нельзя изменить или удалить'; end $$;
create trigger journal_signoffs_no_update before update or delete on journal_signoffs for each row e
xecute function journal_signoffs_immutable();

do $$
declare t text;
begin
  foreach t in array array['journal_entries', 'journal_signoffs'] loop
    execute format('alter table %I enable row level security', t);
    execute format('create policy %I on %I for all to ceh_app using (organization_id = app_org_id()
and app_is_member(organization_id)) with check (organization_id = app_org_id() and app_is_member(org
anization_id))', t || '_tenant', t);
  end loop;
end $$;

grant select, insert on journal_entries, journal_signoffs to ceh_app;

// ===== migrations/006_haccp_capa_evidence.sql =====
-- ЦЕХ — этап 5 ТЗ v2 (часть 2): план ХАССП, CAPA, доказательства.
--
-- ТЗ v1 §10: описание процесса с подтверждением на месте; анализ биологических, химических, физичес
ких и иных
-- опасностей по этапам; PRP/OPRP/CCP назначаются по анализу опасностей, не автоматически; план мони
торинга:
-- что, где, как, частота, предел, прибор, ответственный, запись и реакция. Утверждённая версия план
а не правится —
-- изменения идут новой версией. ТЗ v2 §12: отклонение → CAPA и проверка эффективности; доказательст
ва неизменяемы.

create table haccp_plans (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  version         integer not null,
  status          text not null default 'draft' check (status in ('draft', 'approved', 'superseded')
),
  scope           text not null,                      -- продукция, процессы, площадки
```

```
  team            text,                                -- группа ХАССП
  based_on_id     uuid references haccp_plans(id),
  change_reason   text,
  created_by      uuid not null references users(id),
  created_at      timestamptz not null default now(),
  approved_by     uuid references users(id),
  approved_at     timestamptz,
  approval_note   text,
  unique (organization_id, version),
  check (status = 'draft' or (approved_by is not null and approved_at is not null)),
  check (version = 1 or change_reason is not null)
);
create unique index haccp_plans_one_draft on haccp_plans(organization_id) where status = 'draft';
create unique index haccp_plans_one_approved on haccp_plans(organization_id) where status = 'approve
d';

create table haccp_steps (
  id                uuid primary key default gen_random_uuid(),
  organization_id   uuid not null references organizations(id),
  plan_id           uuid not null references haccp_plans(id),
  position          integer not null,
  name              text not null,
  description       text,
  confirmed_by      uuid references users(id),        -- технологическая схема подтверждена на месте
  confirmed_at      timestamptz,
  confirmation_note text
);

create table haccp_hazards (
  id               uuid primary key default gen_random_uuid(),
  organization_id  uuid not null references organizations(id),
  plan_id          uuid not null references haccp_plans(id),
  step_id          uuid not null references haccp_steps(id),
  code             text not null,                     -- О-01, О-02 … в пределах плана
  hazard_type      text not null check (hazard_type in ('biological', 'chemical', 'physical', 'aller
gen', 'radiological', 'other')),
  description      text not null,
  requirement_id   uuid references reg_requirements(id),
  likelihood       smallint check (likelihood between 1 and 3),   -- оценивает человек; неизвестно ≠
 0
  severity         smallint check (severity between 1 and 3),
  control_measures text,
  answers          jsonb not null default '{}',        -- ответы на вопросы дерева решений
  proposal         text check (proposal in ('ccp', 'not_ccp', 'modify_step')),
  class            text check (class in ('prp', 'oprp', 'ccp')),
  class_rationale  text,
  classified_by    uuid references users(id),
  classified_at    timestamptz,
  check (class is null or (class_rationale is not null and classified_by is not null))
);
```

```
create table haccp_monitoring (
  id                uuid primary key default gen_random_uuid(),
  organization_id   uuid not null references organizations(id),
  plan_id           uuid not null references haccp_plans(id),
  hazard_id         uuid not null references haccp_hazards(id),
  what              text not null,
  where_text        text,
  method            text not null,
  frequency         text not null,
  critical_limit    text,
  device            text,
  responsible_role  text not null,
  record_form       text check (record_form is null or record_form ~ '^J\d{2}$'),
  corrective_action text not null,
  control_rule_id   uuid references control_rules(id),
  unique (hazard_id)
);

-- Утверждённый и заменённый план не меняется: изменения — новой версией.
create function haccp_locked() returns trigger language plpgsql as $$
declare st text;
begin
  select status into st from haccp_plans where id = coalesce(new.plan_id, old.plan_id);
  if st <> 'draft' then raise exception 'План ХАССП утверждён — изменения вносятся новой версией'; e
nd if;
  return coalesce(new, old);
end $$;
create trigger haccp_steps_lock before insert or update or delete on haccp_steps for each row execut
e function haccp_locked();
create trigger haccp_hazards_lock before insert or update or delete on haccp_hazards for each row ex
ecute function haccp_locked();
create trigger haccp_monitoring_lock before insert or update or delete on haccp_monitoring for each
row execute function haccp_locked();

create function haccp_plan_guard() returns trigger language plpgsql as $$
begin
  if tg_op = 'DELETE' then raise exception 'План ХАССП не удаляется'; end if;
  if old.status = 'superseded' or (old.status = 'approved' and (new.status <> 'superseded' or new.sc
ope is distinct from old.scope or new.team is distinct from old.team)) then
    raise exception 'План ХАССП утверждён — изменения вносятся новой версией';
  end if;
  return new;
end $$;
create trigger haccp_plans_guard before update or delete on haccp_plans for each row execute functio
n haccp_plan_guard();

-- CAPA: корректирующие и предупреждающие действия с проверкой эффективности.
create table capas (
  id                    uuid primary key default gen_random_uuid(),
```

```
  organization_id        uuid not null references organizations(id),
  site_id                uuid references sites(id),
  incident_id            uuid references incidents(id),
  title                  text not null,
  problem                text not null,
  risk                   text,
  root_cause             text,
  corrective_action      text,
  preventive_action      text,
  effectiveness_criterion text,
  responsible_user_id    uuid references users(id),
  due_at                 date,
  check_due_at           date,                 -- когда проверять эффективность
  status                 text not null default 'open' check (status in ('open', 'in_progress', 'veri
fication', 'closed')),
  done_note              text,
  done_by                uuid references users(id),
  done_at                timestamptz,
  verified_by            uuid references users(id),
  verified_at            timestamptz,
  verification_note      text,
  created_by             uuid references users(id),
  created_at             timestamptz not null default now(),
  check (status in ('open', 'in_progress') or (root_cause is not null and corrective_action is not n
ull and effectiveness_criterion is not null and done_at is not null)),
  check (status <> 'closed' or (verified_by is not null and verified_at is not null and verified_by
<> done_by))
);
create unique index capas_incident_uniq on capas(incident_id) where incident_id is not null;

create table capa_events (
  id              bigserial primary key,
  organization_id uuid not null references organizations(id),
  capa_id         uuid not null references capas(id),
  user_id         uuid references users(id),
  event           text not null,
  note            text,
  at              timestamptz not null default now()
);

-- Доказательства: файл неизменяем, хранится с хешем; к записи — только добавлять.
create table evidence_files (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  site_id         uuid references sites(id),
  entity_type     text not null,
  entity_id       uuid not null,
  filename        text not null,
  mime            text not null,
  size_bytes      integer not null check (size_bytes > 0),
```

```
  sha256          text not null,
  content         bytea not null,
  note            text,
  uploaded_by     uuid not null references users(id),
  uploaded_at     timestamptz not null default now()
);
create index evidence_entity_idx on evidence_files(organization_id, entity_type, entity_id);

create function evidence_immutable() returns trigger language plpgsql as $$
begin raise exception 'Доказательство нельзя изменить или удалить'; end $$;
create trigger evidence_no_update before update or delete on evidence_files for each row execute fun
ction evidence_immutable();
create function capa_events_immutable() returns trigger language plpgsql as $$
begin raise exception 'История CAPA не изменяется'; end $$;
create trigger capa_events_no_update before update or delete on capa_events for each row execute fun
ction capa_events_immutable();

do $$
declare t text;
begin
  foreach t in array array['haccp_plans', 'haccp_steps', 'haccp_hazards', 'haccp_monitoring', 'capas
', 'capa_events', 'evidence_files'] loop
    execute format('alter table %I enable row level security', t);
    execute format('create policy %I on %I for all to ceh_app using (organization_id = app_org_id()
and app_is_member(organization_id)) with check (organization_id = app_org_id() and app_is_member(org
anization_id))', t || '_tenant', t);
  end loop;
end $$;

grant select, insert, update on haccp_plans, capas to ceh_app;
grant select, insert, update, delete on haccp_steps, haccp_hazards, haccp_monitoring to ceh_app;
grant select, insert on capa_events, evidence_files to ceh_app;
grant usage on sequence capa_events_id_seq to ceh_app;

// ===== migrations/007_recipes_costing.sql =====
-- ЦЕХ — этап 6 ТЗ v2 (часть 1): блюда и полуфабрикаты, ТТК с версиями и утверждением, себестоимость
, КБЖУ,
-- архив ТТК (560 карт) как справочник.
--
-- ТЗ v1 §9, §12: ТТК утверждает человек; при производстве и продаже — списание по утверждённой верс
ии.
-- Неизвестная цена ≠ 0: себестоимость без цены помечается неполной. КБЖУ — из справочника MON CHEF
со статусом
-- проверки (блокер Б-5: все значения «REVIEW»).

-- Цена из документа поставки — за единицу партии.
alter table receipt_lines add column unit_price numeric check (unit_price is null or unit_price >= 0
);
alter table lots add column unit_price numeric check (unit_price is null or unit_price >= 0);
```

```
-- Архив ТТК: справочник уровня платформы (как reference_products), только чтение для организаций.
create table ttk_archive (
  id             uuid primary key default gen_random_uuid(),
  source_path    text not null unique,
  category_path  text[] not null default '{}',
  name           text not null,
  yield_raw      text,
  yield_g        numeric,
  yield_source   text,
  basis          text,
  format         text,
  lines          jsonb not null default '[]',
  technology     text,
  issues         jsonb not null default '[]',
  imported_at    timestamptz not null default now()
);
grant select on ttk_archive to ceh_app;

create table dishes (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  name            text not null,
  kind            text not null default 'dish' check (kind in ('dish', 'semi')),   -- блюдо или полу
фабрикат
  category        text,
  sale_price      numeric check (sale_price is null or sale_price >= 0),
  product_id      uuid references products(id),        -- п/ф как складская позиция (выпуск партии)
  archive_id      uuid references ttk_archive(id),
  status          text not null default 'active' check (status in ('active', 'archived')),
  created_by      uuid references users(id),
  created_at      timestamptz not null default now()
);
create unique index dishes_name_uniq on dishes(organization_id, lower(name)) where status = 'active'
;

create table ttk_versions (
  id                 uuid primary key default gen_random_uuid(),
  organization_id    uuid not null references organizations(id),
  dish_id            uuid not null references dishes(id),
  version            integer not null,
  status             text not null default 'draft' check (status in ('draft', 'review', 'approved',
'archived')),
  yield_g            numeric check (yield_g is null or yield_g > 0),     -- выход закладки
  portion_g          numeric check (portion_g is null or portion_g > 0), -- выход порции (для блюда)
  technology         text,
  storage_conditions text,
  shelf_life         text,
  quality            text,
  allergens          text,
  source             text not null default 'Собственная разработка',
```

```
  change_reason      text,
  created_by         uuid not null references users(id),
  created_at         timestamptz not null default now(),
  submitted_by       uuid references users(id),
  submitted_at       timestamptz,
  approved_by        uuid references users(id),
  approved_at        timestamptz,
  archived_at        timestamptz,
  return_note        text,
  unique (dish_id, version),
  check (status not in ('approved', 'archived') or (approved_by is not null and approved_at is not n
ull))
);
create unique index ttk_one_approved on ttk_versions(dish_id) where status = 'approved';
create unique index ttk_one_open on ttk_versions(dish_id) where status in ('draft', 'review');

create table ttk_lines (
  id              uuid primary key default gen_random_uuid(),
  organization_id uuid not null references organizations(id),
  version_id      uuid not null references ttk_versions(id),
  position        integer not null,
  name_raw        text not null,                      -- как в карте
  product_id      uuid references products(id),
  sub_dish_id     uuid references dishes(id),         -- полуфабрикат / вложенная ТТК
  unit            text not null check (unit in ('кг', 'г', 'л', 'мл', 'шт', 'порция')),
  brutto          numeric check (brutto is null or brutto >= 0),
  netto           numeric check (netto is null or netto >= 0),
  note            text,
  check (product_id is null or sub_dish_id is null)
);
create index ttk_lines_version_idx on ttk_lines(version_id, position);

-- Сопоставление «название в карте → товар» запоминается на организацию: сопоставили раз — применяет
ся везде.
create table ingredient_aliases (
  organization_id uuid not null references organizations(id),
  name_norm       text not null,
  product_id      uuid references products(id),
  sub_dish_id     uuid references dishes(id),
  created_by      uuid not null references users(id),
  created_at      timestamptz not null default now(),
  primary key (organization_id, name_norm),
  check ((product_id is null) <> (sub_dish_id is null))
);

-- Утверждённая и архивная версия не меняется: изменения — новой версией.
create function ttk_lines_locked() returns trigger language plpgsql as $$
declare st text;
begin
  select status into st from ttk_versions where id = coalesce(new.version_id, old.version_id);
```

… страницы 26–326 не приводятся …

```
                <div className="small">можно расходовать: <b>{n(r.available)} {r.unit}</b>{r.next_ex
piry ? ` · ближайший срок ${d(r.next_expiry, org.timezone)}` : ''}</div>
                <div className="small muted">партий: {r.lots}</div></div>
              <div className="row" style={{ justifyContent: 'flex-end' }}>
                {Number(r.soon) > 0 && <span className="badge warn">скоро: {n(r.soon)}</span>}
                {Number(r.quarantine) > 0 && <span className="badge crit">карантин: {n(r.quarantine)
}</span>}
                {Number(r.unconfirmed) > 0 && <span className="badge warn">срок не подтв.: {n(r.unco
nfirmed)}</span>}
                {Number(r.unusable) > 0 && <span className="badge crit">нельзя: {n(r.unusable)}</spa
n>}
              </div>
            </Link>
          ))}</div>
        )}
      </div>
    </>
  );
}

// ===== src/app/app/[orgId]/stop-list/page.tsx =====
'use client';
import { useCallback, useEffect, useState } from 'react';
import Link from 'next/link';
import { useOrg } from '@/lib/org';
import { SitePicker } from '@/components/OrgNav';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';
import { dt } from '@/lib/fmt';

interface Stop { id: string; dish_id: string; dish: string; reason: string; source: string; created_
at: string; who: string | null }
interface Dish { id: string; name: string; kind: string; sale_price: string | null }

/** Стоп-лист площадки: автоматический — по годному остатку и утверждённой ТТК, ручной — с причиной.
 */
export default function StopListPage() {
  const { org, siteId, can } = useOrg();
  const [stops, setStops] = useState<Stop[] | null>(null);
  const [dishes, setDishes] = useState<Dish[]>([]);
  const [f, setF] = useState({ dishId: '', reason: '' });
  const { busy, error, run } = useSubmit();
  const load = useCallback(async () => {
    if (!siteId) return;
    setStops(await api<Stop[]>(`/orgs/${org.id}/sites/${siteId}/stop-list`));
    setDishes((await api<Dish[]>(`/orgs/${org.id}/dishes`)).filter((d) => d.kind === 'dish'));
  }, [org.id, siteId]);
  useEffect(() => { load(); }, [load]);
  return (
    <>
```

```
      <p className="small"><Link href={`/app/${org.id}`}>← Управление</Link></p>
      <SitePicker />
      <h1>Стоп-лист</h1>
      <p className="small muted">Блюдо попадает в стоп само, если по утверждённой ТТК из годного ост
атка не собрать ни одной порции; снимается само, когда сырьё пришло. Без утверждённой ТТК автоматиче
ский стоп не ставится.</p>
      {error && <div className="err">{error}</div>}
      <div className="card">
        {!stops ? <p className="muted">Загрузка…</p> : stops.length === 0 ? <p className="muted">Сто
п-лист пуст.</p> : (
          <ul className="list">{stops.map((s) => (
            <li key={s.id} className="between">
              <span><b>{s.dish}</b><div className="small">{s.reason}</div>
                <div className="small muted">{s.source === 'auto' ? 'автоматически' : s.who}, {dt(s.
created_at, org.timezone)}</div></span>
              {can('pos.stop') && <button className="btn small secondary" disabled={busy} onClick={(
) => run(async () => { await api(`/orgs/${org.id}/stop-list/${s.id}`, { method: 'DELETE' }); await l
oad(); })}>Снять</button>}
            </li>
          ))}</ul>
        )}
      </div>
      {can('pos.stop') && (
        <form className="card" onSubmit={(e) => { e.preventDefault(); run(async () => { await api(`/
orgs/${org.id}/stop-list`, { body: { siteId, ...f } }); setF({ dishId: '', reason: '' }); await load
(); }); }}>
          <h2>Поставить в стоп</h2>
          <label htmlFor="sd">Блюдо</label>
          <select id="sd" required value={f.dishId} onChange={(e) => setF({ ...f, dishId: e.target.v
alue })}><option value="">— выберите —</option>{dishes.map((d) => <option key={d.id} value={d.id}>{d
.name}</option>)}</select>
          <label htmlFor="sr">Причина</label>
          <input id="sr" required value={f.reason} onChange={(e) => setF({ ...f, reason: e.target.va
lue })} placeholder="например, сломалась фритюрница" />
          <button className="btn block" disabled={busy}>В стоп</button>
        </form>
      )}
    </>
  );
}

// ===== src/app/app/[orgId]/tasks/page.tsx =====
'use client';
import { useCallback, useEffect, useState } from 'react';
import Link from 'next/link';
import { useOrg } from '@/lib/org';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';
import { dt } from '@/lib/fmt';
```

```
interface Task { id: string; title: string; details: string | null; due_at: string | null; overdue:
boolean; href: string; action: string; manual: boolean; site_name: string | null }

export default function TasksPage() {
  const { org, role } = useOrg();
  const manager = ['owner', 'enterprise_admin', 'director', 'chef', 'haccp_lead'].includes(role);
  const [all, setAll] = useState(false);
  const [tasks, setTasks] = useState<Task[] | null>(null);
  const { error, run } = useSubmit();
  const load = useCallback(() => api<Task[]>(`/orgs/${org.id}/tasks${all ? '?all=1' : ''}`).then(set
Tasks), [org.id, all]);
  useEffect(() => { load(); }, [load]);
  return (
    <>
      <h1>Задачи</h1>
      {manager && <div className="chips"><button aria-pressed={!all} onClick={() => setAll(false)}>М
ои</button><button aria-pressed={all} onClick={() => setAll(true)}>Все по предприятию</button></div>
}
      {error && <div className="err">{error}</div>}
      <div className="card">
        {!tasks ? <p className="muted">Загрузка…</p> : tasks.length === 0 ? <p className="muted">Отк
рытых задач нет.</p> : (
          <ul className="list">{tasks.map((t) => (
            <li key={t.id} id={t.id}>
              <div className="between">
                <div><b>{t.title}</b>{t.details && <div className="small muted">{t.details}</div>}
                  <div className="small">{t.site_name}{t.due_at ? ` · срок ${dt(t.due_at, org.timezo
ne)}` : ''}</div></div>
                {t.overdue && <span className="badge crit">просрочено</span>}
              </div>
              <div className="row" style={{ marginTop: 8 }}>
                {!t.manual || t.href.includes('/lots/') || t.href.includes('/incidents/') || t.href.
includes('/objects/')
                  ? <Link className="btn small" href={t.href}>{t.action}</Link>
                  : <button className="btn small" onClick={() => { const r = prompt('Результат выпол
нения:'); if (r) run(async () => { await api(`/orgs/${org.id}/tasks/${t.id}/complete`, { body: { res
ult: r } }); await load(); }); }}>Выполнено</button>}
              </div>
            </li>
          ))}</ul>
        )}
      </div>
    </>
  );
}

// ===== src/app/app/page.tsx =====
'use client';
import { useCallback, useEffect, useState } from 'react';
import Link from 'next/link';
```

```
import { useRouter } from 'next/navigation';
import { AppShell, type Me } from '@/components/AppShell';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';

interface OrgRow { id: string; name: string; status: string; role: string; role_label: string }

const STATUS: Record<string, string> = { DRAFT: 'Черновик', CONFIGURED: 'Настроено', COMPLIANCE_REVI
EW: 'Нормативная проверка', ARCHIVED: 'Архив' };

function Orgs({ me }: { me: Me }) {
  const router = useRouter();
  const [orgs, setOrgs] = useState<OrgRow[] | null>(null);
  const [form, setForm] = useState({ name: '', legalForm: 'ООО', inn: '', region: '', firstSiteName:
 '' });
  const [resent, setResent] = useState(false);
  const { busy, error, run } = useSubmit();
  const load = useCallback(() => api<OrgRow[]>('/orgs').then(setOrgs), []);
  useEffect(() => { load(); }, [load]);
  const set = (k: keyof typeof form) => (e: React.ChangeEvent<HTMLInputElement | HTMLSelectElement>)
 => setForm({ ...form, [k]: e.target.value });

  return (
    <>
      <h1>Мои предприятия</h1>
      {!me.email_verified && (
        <div className="note">
          Подтвердите email <b>{me.email}</b> — ссылка в письме после регистрации. Без подтверждения
 нельзя создать предприятие или принять приглашение.
          <div style={{ marginTop: 8 }}>
            {resent ? <span className="small">Письмо отправлено повторно.</span> :
              <button className="btn secondary small" onClick={async () => { await api('/auth/resend
-verification', { body: {} }); setResent(true); }}>Отправить письмо ещё раз</button>}
          </div>
        </div>
      )}
      <div className="card">
        {orgs === null ? <p className="muted">Загрузка…</p> : orgs.length === 0 ? <p className="mute
d">Пока нет предприятий. Создайте своё или примите приглашение по ссылке из письма.</p> : (
          <ul className="list">
            {orgs.map((o) => (
              <li key={o.id} className="between">
                <div><Link href={`/app/${o.id}`}><b>{o.name}</b></Link><div className="small muted">
{o.role_label}</div></div>
                <span className="badge">{STATUS[o.status] || o.status}</span>
              </li>
            ))}
          </ul>
        )}
      </div>
```

```
      {me.email_verified && (
        <div className="card">
          <h2>Новое предприятие</h2>
          <p className="muted small">Вы станете владельцем. Паспорт предприятия (помещения, цеха, об
орудование) заполняется следующим шагом; до проверки статус — «Черновик».</p>
          <form onSubmit={(e) => { e.preventDefault(); run(async () => {
            const r = await api<{ organization: { id: string } }>('/orgs', { body: { ...form, inn: f
orm.inn || undefined, region: form.region || undefined, firstSiteName: form.firstSiteName || undefin
ed } });
            router.push(`/app/${r.organization.id}`);
          }); }}>
            <label htmlFor="name">Название</label>
            <input id="name" required value={form.name} onChange={set('name')} placeholder="Кафе «Пр
имер»" />
            <div className="grid2">
              <div><label htmlFor="lf">Форма</label>
                <select id="lf" value={form.legalForm} onChange={set('legalForm')}>{['ООО', 'ИП', 'А
О', 'другое'].map((x) => <option key={x}>{x}</option>)}</select></div>
              <div><label htmlFor="inn">ИНН</label><input id="inn" inputMode="numeric" value={form.i
nn} onChange={set('inn')} placeholder="10 или 12 цифр" /></div>
              <div><label htmlFor="region">Регион</label><input id="region" value={form.region} onCh
ange={set('region')} /></div>
              <div><label htmlFor="site">Первая площадка</label><input id="site" value={form.firstSi
teName} onChange={set('firstSiteName')} placeholder="Основная площадка" /></div>
            </div>
            {error && <div className="err" role="alert">{error}</div>}
            <button className="btn block" disabled={busy}>Создать предприятие</button>
          </form>
        </div>
      )}
    </>
  );
}

export default function Page() { return <AppShell>{(me) => <Orgs me={me} />}</AppShell>; }

// ===== src/app/forgot/page.tsx =====
'use client';
import { useState } from 'react';
import Link from 'next/link';
import { AuthShell } from '@/components/AuthShell';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';

export default function Page() {
  const [email, setEmail] = useState('');
  const [sent, setSent] = useState(false);
  const { busy, error, run } = useSubmit();
  return (
    <AuthShell title="Восстановление пароля">
```

```
      {sent ? (
        <><p>Если такой адрес зарегистрирован, на него придёт письмо со ссылкой. Ссылка действует 1
час.</p><Link className="btn block" href="/login">Ко входу</Link></>
      ) : (
        <form onSubmit={(e) => { e.preventDefault(); run(async () => { await api('/auth/password/for
got', { body: { email } }); setSent(true); }); }}>
          <label htmlFor="email">Email</label>
          <input id="email" type="email" autoComplete="email" required value={email} onChange={(e) =
> setEmail(e.target.value)} />
          {error && <div className="err" role="alert">{error}</div>}
          <button className="btn block" disabled={busy}>Отправить ссылку</button>
        </form>
      )}
    </AuthShell>
  );
}

// ===== src/app/globals.css =====
/* Оформление прежней системы ЦЕХ (Apps Script): жёлтый, тёмно-коричневый, бежевый фон, белые карточ
ки с тенью. */
:root {
  --yellow: #F5A623; --brown: #3B2A20; --beige: #F1E6D8; --burgundy: #7A1F2B; --black: #1A1410;
  --bg: var(--beige); --surface: #ffffff; --ink: var(--black); --muted: #8a7a6a; --line: #e6dccf;
  --brand: var(--brown); --accent: var(--yellow); --accent-ink: var(--black);
  --ok: #16A344; --ok-bg: #e4f5ea; --info: #388276; --warn: #F59E08; --warn-ink: #8a5300; --warn-bg:
 #fff3dc;
  --crit: #DC2626; --crit-bg: #fde8e8;
  --radius: 14px; --tap: 46px; --shadow: 0 1px 4px rgba(0,0,0,.08);
  color-scheme: light;
}
* { box-sizing: border-box; }
html, body { margin: 0; background: var(--bg); color: var(--ink); }
body { font: 15px/1.45 -apple-system, 'Segoe UI', Roboto, Arial, sans-serif; -webkit-text-size-adjus
t: 100%; }
a { color: inherit; }
h1 { font-size: 22px; margin: 0 0 12px; color: var(--brown); }
h2 { font-size: 16px; margin: 0 0 10px; }
h3 { font-size: 15px; margin: 0 0 8px; }
.wrap { max-width: 720px; margin: 0 auto; padding: 12px 12px 24px; }
.narrow { max-width: 400px; margin: 0 auto; padding: 28px 16px; }
.card { background: var(--surface); border-radius: var(--radius); padding: 14px; margin-bottom: 12px
; box-shadow: var(--shadow); }
.muted { color: var(--muted); }
.small { font-size: 13px; }
label { display: block; font-size: 13px; font-weight: 600; margin: 10px 0 5px; color: var(--brown);
}
input, select, textarea { width: 100%; min-height: var(--tap); padding: 10px 11px; font: inherit; fo
nt-size: 15px; color: var(--ink); background: var(--surface);
  border: 1px solid #ddd; border-radius: 10px; }
input:focus, select:focus, textarea:focus, button:focus-visible, a:focus-visible { outline: 2px soli
```

```
d var(--yellow); outline-offset: 1px; }
.btn { display: inline-flex; align-items: center; justify-content: center; gap: 8px; min-height: var
(--tap); padding: 0 16px; border-radius: 10px;
  border: 0; font-family: inherit; font-size: 15px; font-weight: 700; line-height: 1.1; cursor: poin
ter; background: var(--yellow); color: var(--black); text-decoration: none; }
.btn.block { width: 100%; margin-top: 14px; }
.btn.secondary { background: var(--surface); color: var(--brown); border: 1px solid var(--brown); }
.btn.danger { background: var(--burgundy); color: #fff; }
.btn.small { min-height: 36px; padding: 0 12px; font-size: 13px; }
.btn:disabled { opacity: .55; cursor: default; }
.err { color: var(--crit); background: var(--crit-bg); border-radius: 10px; padding: 10px 12px; marg
in-top: 12px; }
.note { background: var(--warn-bg); color: var(--ink); border-left: 3px solid var(--warn); border-ra
dius: 10px; padding: 10px 12px; margin: 12px 0; }
.okbox { background: var(--ok-bg); border-left: 3px solid var(--ok); border-radius: 10px; padding: 1
0px 12px; margin: 12px 0; }
.badge { display: inline-block; padding: 3px 8px; border-radius: 20px; font-size: 11px; font-weight:
 700; white-space: nowrap; background: var(--beige); color: var(--brown); }
.badge.ok { background: var(--ok); color: #fff; } .badge.warn { background: var(--warn); color: #fff
; } .badge.crit { background: var(--crit); color: #fff; }
.row { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; }
.between { display: flex; justify-content: space-between; align-items: center; gap: 10px; flex-wrap:
 wrap; }
.grid2 { display: grid; grid-template-columns: 1fr; gap: 0 10px; }
@media (min-width: 560px) { .grid2 { grid-template-columns: 1fr 1fr; } }

/* Шапка: ☰, логотип, справа имя и роль (нажатие — выход) */
.topbar { position: sticky; top: 0; z-index: 15; background: var(--brown); color: #fff; display: fle
x; align-items: center; gap: 10px;
  padding: 10px max(14px, env(safe-area-inset-right)) 10px max(14px, env(safe-area-inset-left)); }
.topbar .logo { color: var(--yellow); flex: 1; display: flex; }
.hamburger { background: none; border: 0; color: var(--yellow); font-size: 24px; line-height: 1; pad
ding: 4px 8px 4px 0; cursor: pointer; min-height: 40px; }
.who { background: none; border: 0; color: #fff; font: inherit; font-size: 12px; text-align: right;
cursor: pointer; display: flex; flex-direction: column; opacity: .9; padding: 2px 0; }
.who .role { color: #cdbba8; font-size: 11px; }

/* Выдвижное меню разделов */
.menu-overlay { position: fixed; inset: 0; background: rgba(0,0,0,.4); z-index: 20; opacity: 0; poin
ter-events: none; transition: opacity .2s; }
.menu-overlay.open { opacity: 1; pointer-events: auto; }
.sidemenu { position: fixed; top: 0; left: 0; bottom: 0; width: 78%; max-width: 300px; background: v
ar(--surface); z-index: 21; overflow-y: auto;
  transform: translateX(-100%); visibility: hidden; transition: transform .25s, visibility 0s .25s;
padding: 16px 0 calc(16px + env(safe-area-inset-bottom)); }
.sidemenu.open { transform: translateX(0); visibility: visible; transition: transform .25s; box-shad
ow: 4px 0 18px rgba(0,0,0,.18); }
.menu-head { color: var(--brown); padding: 0 16px 12px; border-bottom: 1px solid #eee; margin-bottom
: 6px; display: flex; flex-direction: column; gap: 6px; }
.menu-head .small { color: var(--muted); }
```

```
.menu-group { padding: 10px 16px 4px; font-size: 11px; text-transform: uppercase; letter-spacing: .5
px; color: #a99783; }
.menu-item { display: block; padding: 12px 16px; font-size: 15px; color: var(--black); text-decorati
on: none; }
.menu-item[aria-current="page"] { background: var(--beige); color: var(--brown); font-weight: 700; }

/* Вход: логотип на белой плашке, маскот подпрыгивает и моргает */
.brandblock { color: var(--brand); display: flex; justify-content: center; margin: 8px 0 18px; }
.plate { background: #fff; border-radius: 14px; box-shadow: 0 10px 24px -12px rgba(0,0,0,.35); paddi
ng: 18px 24px; color: var(--brown); }
.ceh-anim .ceh-mascot { animation: cehBounce 1.5s ease-in-out infinite; transform-origin: 232px 300p
x; }
.ceh-anim .ceh-eye { animation: cehBlink 3.6s ease-in-out infinite; transform-origin: center; transf
orm-box: fill-box; }
.ceh-anim .ceh-letter { opacity: 0; animation: cehLetterIn .5s cubic-bezier(.2,.8,.3,1) forwards; }
.ceh-anim .ceh-letter.e { animation-delay: .5s; } .ceh-anim .ceh-letter.h { animation-delay: .75s; }
@keyframes cehBounce { 0%,100% { transform: translateY(0) scaleY(1); } 30% { transform: translateY(-
14px) scaleY(1.05); } 55% { transform: translateY(0) scaleY(.93); } 75% { transform: translateY(-4px
) scaleY(1.02); } }
@keyframes cehBlink { 0%,90%,100% { transform: scaleY(1); } 94% { transform: scaleY(.12); } }
@keyframes cehLetterIn { from { opacity: 0; transform: translateX(20px); } to { opacity: 1; transfor
m: translateX(0); } }
@media (prefers-reduced-motion: reduce) { .ceh-anim .ceh-mascot, .ceh-anim .ceh-eye { animation: non
e; } .ceh-anim .ceh-letter { animation: none; opacity: 1; } }
.tabs { display: flex; gap: 6px; overflow-x: auto; margin: 4px 0 14px; padding-bottom: 2px; }
.tabs button { flex: none; min-height: 38px; padding: 0 14px; border-radius: 20px; border: 1px solid
 #ddd; background: var(--surface); color: var(--brown); font-family: inherit; font-size: 14px; font-
weight: 600; cursor: pointer; }
.tabs button[aria-selected="true"] { background: var(--brown); color: #fff; border-color: var(--brow
n); }
.list { list-style: none; margin: 0; padding: 0; }
.list li { padding: 12px 0; border-top: 1px solid var(--line); }
.list li:first-child { border-top: 0; }
.tbl { width: 100%; border-collapse: collapse; font-size: 14px; }
.tbl th, .tbl td { text-align: left; padding: 8px 6px; border-top: 1px solid var(--line); vertical-a
lign: top; }
.tbl th { font-size: 12px; color: var(--muted); font-weight: 600; border-top: 0; }
.scroll { overflow-x: auto; }
.center { text-align: center; }

/* ——— Этап 2: экран сотрудника ——— */
.tiles { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-bottom: 14px; }
.tile { display: flex; flex-direction: column; justify-content: space-between; min-height: 100px; pa
dding: 14px; border-radius: var(--radius);
  background: var(--surface); border: 1.5px solid transparent; box-shadow: var(--shadow); text-decor
ation: none; color: var(--ink); font-weight: 700; font-size: 17px; }
.tile .sub { font-size: 13px; font-weight: 500; color: var(--muted); }
.tile.primary { background: var(--accent); border-color: var(--accent); color: var(--accent-ink); }
.tile.primary .sub { color: var(--accent-ink); opacity: .8; }
.tile.alert { border-color: var(--crit); }
```

```
.tile.alert .sub { color: var(--crit); font-weight: 700; }
.objrow { display: flex; align-items: center; justify-content: space-between; gap: 10px; padding: 12
px 0; border-top: 1px solid var(--line); text-decoration: none; color: var(--ink); }
.objrow:first-child { border-top: 0; }
.big-input { font-size: 32px; font-weight: 700; text-align: center; min-height: 64px; }
.prob { border-left: 6px solid var(--line); }
.prob.crit { border-left-color: var(--crit); }
.prob.warn { border-left-color: var(--accent); }
.kv { display: grid; grid-template-columns: max-content 1fr; gap: 4px 12px; font-size: 14px; margin:
 8px 0; }
.kv dt { color: var(--muted); }
.kv dd { margin: 0; }
.chips { display: flex; gap: 6px; flex-wrap: wrap; margin-bottom: 12px; }
.chips button { min-height: 34px; padding: 0 12px; border-radius: 20px; border: 1px solid #ddd; back
ground: var(--surface); color: var(--brown); font-family: inherit; font-size: 13px; font-weight: 600
; cursor: pointer; }
.chips button[aria-pressed="true"] { background: var(--brown); color: #fff; border-color: var(--brow
n); }
.seg { display: grid; grid-template-columns: 1fr 1fr; gap: 6px; }
.seg button { min-height: 44px; border-radius: 10px; border: 1px solid #ddd; background: var(--surfa
ce); color: var(--ink); font-family: inherit; font-size: 14px; font-weight: 600; cursor: pointer; }
.seg button[aria-pressed="true"].yes { background: var(--ok-bg); border-color: var(--ok); color: var
(--ok); }
.seg button[aria-pressed="true"].no { background: var(--crit-bg); border-color: var(--crit); color:
var(--crit); }
.timeline { list-style: none; padding: 0; margin: 0; border-left: 2px solid var(--line); }
.timeline li { padding: 4px 0 10px 12px; position: relative; font-size: 14px; }
.timeline li::before { content: ''; position: absolute; left: -6px; top: 9px; width: 10px; height: 1
0px; border-radius: 50%; background: var(--brand); }

// ===== src/app/invite/page.tsx =====
'use client';
import { Suspense, useEffect, useState } from 'react';
import { useRouter, useSearchParams } from 'next/navigation';
import Link from 'next/link';
import { AuthShell } from '@/components/AuthShell';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';

interface Me { user: { email: string; full_name: string; email_verified: boolean } | null }

function Invite() {
  const token = useSearchParams().get('token') || '';
  const router = useRouter();
  const [me, setMe] = useState<Me | null>(null);
  const { busy, error, run } = useSubmit();
  useEffect(() => { api<Me>('/auth/me').then(setMe).catch(() => setMe({ user: null })); }, []);
  const back = `/invite?token=${encodeURIComponent(token)}`;
  return (
    <AuthShell title="Приглашение">
```

```
      {!me && <p>Загрузка…</p>}
      {me && !me.user && (
        <>
          <p>Чтобы принять приглашение, войдите или зарегистрируйтесь <b>на тот email, на который пр
ишло письмо</b>.</p>
          <Link className="btn block" href={`/login?next=${encodeURIComponent(back)}`}>Войти</Link>
          <Link className="btn block secondary" href="/register">Зарегистрироваться</Link>
        </>
      )}
      {me?.user && (
        <>
          <p>Вы вошли как <b>{me.user.email}</b>.</p>
          {error && <div className="err" role="alert">{error}</div>}
          <button className="btn block" disabled={busy} onClick={() => run(async () => {
            const r = await api<{ organizationId: string }>('/invitations/accept', { body: { token }
 });
            router.replace(`/app/${r.organizationId}`);
          })}>Принять приглашение</button>
        </>
      )}
    </AuthShell>
  );
}

export default function Page() { return <Suspense><Invite /></Suspense>; }

// ===== src/app/layout.tsx =====
import type { Metadata, Viewport } from 'next';
import './globals.css';

export const metadata: Metadata = { title: 'ЦЕХ', description: 'Единая система управления общепитом'
 };
export const viewport: Viewport = { width: 'device-width', initialScale: 1, themeColor: '#3b2a1e' };

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="ru">
      <body>{children}</body>
    </html>
  );
}

// ===== src/app/login/page.tsx =====
'use client';
import { Suspense, useState } from 'react';
import { useRouter, useSearchParams } from 'next/navigation';
import Link from 'next/link';
import { AuthShell } from '@/components/AuthShell';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';
```

```
function LoginForm() {
  const router = useRouter();
  const next = useSearchParams().get('next') || '/app';
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const { busy, error, run } = useSubmit();
  return (
    <AuthShell title="Вход">
      <form onSubmit={(e) => { e.preventDefault(); run(async () => { await api('/auth/login', { body
: { email, password } }); router.replace(next.startsWith('/') ? next : '/app'); }); }}>
        <label htmlFor="email">Email</label>
        <input id="email" type="email" autoComplete="email" inputMode="email" required value={email}
 onChange={(e) => setEmail(e.target.value)} />
        <label htmlFor="password">Пароль</label>
        <input id="password" type="password" autoComplete="current-password" required value={passwor
d} onChange={(e) => setPassword(e.target.value)} />
        {error && <div className="err" role="alert">{error}</div>}
        <button className="btn block" disabled={busy}>{busy ? 'Входим…' : 'Войти'}</button>
      </form>
      <p className="between small" style={{ marginTop: 16 }}>
        <Link href="/register">Регистрация</Link>
        <Link href="/forgot">Забыли пароль?</Link>
      </p>
    </AuthShell>
  );
}

export default function Page() { return <Suspense><LoginForm /></Suspense>; }

// ===== src/app/page.tsx =====
import { redirect } from 'next/navigation';

export default function Home() {
  redirect('/app');
}

// ===== src/app/register/page.tsx =====
'use client';
import { useState } from 'react';
import Link from 'next/link';
import { AuthShell } from '@/components/AuthShell';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';

export default function Page() {
  const [form, setForm] = useState({ fullName: '', email: '', password: '' });
  const [sent, setSent] = useState(false);
  const { busy, error, run } = useSubmit();
  const set = (k: keyof typeof form) => (e: React.ChangeEvent<HTMLInputElement>) => setForm({ ...for
```

```
m, [k]: e.target.value });
  if (sent) {
    return (
      <AuthShell title="Проверьте почту">
        <p>Мы отправили письмо на <b>{form.email}</b>. Откройте ссылку в письме, чтобы подтвердить а
дрес — после этого можно создать предприятие.</p>
        <p className="muted small">Письмо не пришло за несколько минут — проверьте папку «Спам».</p>
        <Link className="btn block" href="/login">Перейти ко входу</Link>
      </AuthShell>
    );
  }
  return (
    <AuthShell title="Регистрация">
      <form onSubmit={(e) => { e.preventDefault(); run(async () => { await api('/auth/register', { b
ody: form }); setSent(true); }); }}>
        <label htmlFor="fullName">Имя и фамилия</label>
        <input id="fullName" autoComplete="name" required value={form.fullName} onChange={set('fullN
ame')} />
        <label htmlFor="email">Email</label>
        <input id="email" type="email" autoComplete="email" inputMode="email" required value={form.e
mail} onChange={set('email')} />
        <label htmlFor="password">Пароль</label>
        <input id="password" type="password" autoComplete="new-password" required minLength={10} val
ue={form.password} onChange={set('password')} />
        <p className="muted small">Не короче 10 символов, буквы и цифры.</p>
        <p className="muted small">Регистрируясь, вы соглашаетесь с политикой обработки персональных
 данных организации-оператора.</p>
        {error && <div className="err" role="alert">{error}</div>}
        <button className="btn block" disabled={busy}>{busy ? 'Отправляем…' : 'Зарегистрироваться'}<
/button>
      </form>
      <p className="small" style={{ marginTop: 16 }}>Уже есть учётная запись? <Link href="/login">Во
йти</Link></p>
    </AuthShell>
  );
}

// ===== src/app/reset/page.tsx =====
'use client';
import { Suspense, useState } from 'react';
import { useSearchParams } from 'next/navigation';
import Link from 'next/link';
import { AuthShell } from '@/components/AuthShell';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';

function Reset() {
  const token = useSearchParams().get('token') || '';
  const [password, setPassword] = useState('');
  const [done, setDone] = useState(false);
```

```
  const { busy, error, run } = useSubmit();
  return (
    <AuthShell title="Новый пароль">
      {done ? (
        <><div className="okbox">Пароль изменён. Все открытые сеансы на других устройствах закрыты.<
/div><Link className="btn block" href="/login">Войти</Link></>
      ) : (
        <form onSubmit={(e) => { e.preventDefault(); run(async () => { await api('/auth/password/res
et', { body: { token, password } }); setDone(true); }); }}>
          <label htmlFor="password">Новый пароль</label>
          <input id="password" type="password" autoComplete="new-password" required minLength={10} v
alue={password} onChange={(e) => setPassword(e.target.value)} />
          <p className="muted small">Не короче 10 символов, буквы и цифры.</p>
          {error && <div className="err" role="alert">{error}</div>}
          <button className="btn block" disabled={busy}>Сохранить пароль</button>
        </form>
      )}
    </AuthShell>
  );
}

export default function Page() { return <Suspense><Reset /></Suspense>; }

// ===== src/app/verify/page.tsx =====
'use client';
import { Suspense, useEffect, useRef, useState } from 'react';
import { useSearchParams } from 'next/navigation';
import Link from 'next/link';
import { AuthShell } from '@/components/AuthShell';
import { api } from '@/lib/api';

function Verify() {
  const token = useSearchParams().get('token');
  const [state, setState] = useState<'wait' | 'ok' | 'err'>('wait');
  const [msg, setMsg] = useState('');
  const once = useRef(false);
  useEffect(() => {
    if (once.current) return; once.current = true;
    api('/auth/verify', { body: { token } }).then(() => setState('ok')).catch((e) => { setMsg(e.mess
age); setState('err'); });
  }, [token]);
  return (
    <AuthShell title="Подтверждение email">
      {state === 'wait' && <p>Проверяем ссылку…</p>}
      {state === 'ok' && <><div className="okbox">Email подтверждён.</div><Link className="btn block
" href="/app">Продолжить</Link></>}
      {state === 'err' && <><div className="err" role="alert">{msg}</div><Link className="btn block
secondary" href="/app">Запросить новую ссылку</Link></>}
    </AuthShell>
  );
```

```
}

export default function Page() { return <Suspense><Verify /></Suspense>; }

// ===== src/components/AiAnswer.tsx =====
'use client';
import { useState } from 'react';
import Link from 'next/link';
import { useOrg } from '@/lib/org';
import { useSubmit } from '@/components/useSubmit';
import { api } from '@/lib/api';
import { d } from '@/lib/fmt';

export interface Citation { id: string; code: string; title: string; clause: string; edition: string
; state_label: string; last_checked_at: string | null; next_check_at: string | null; text: string; q
uote?: string }
export interface AiResult {
  job_id: string; status: string; message: string | null; model: string | null;
  answer: { found: string; why: string; do_now: string[]; dont: string[]; who_confirms: string; clos
es_when: string; limitations: string; confidence: 'low' | 'medium' | 'high' } | null;
  citations: Citation[]; sources: Citation[]; unsupported: string[]; bad_citations: { source: string
; quote: string; reason: string }[];
}

const CONF: Record<string, [string, string]> = { high: ['высокая', 'ok'], medium: ['средняя', 'warn'
], low: ['низкая', 'crit'] };
const ROLES: [string, string][] = [['haccp_lead', 'Ответственный ХАССП'], ['chef', 'Шеф'], ['line_st
aff', 'Сотрудник цеха'], ['technologist', 'Технолог'], ['storekeeper', 'Кладовщик'], ['director', 'Д
иректор']];

/** Реквизиты источника — обязательная часть нормативной рекомендации (Т3 v2 АТ-07). */
export function CitationCard({ c }: { c: Citation }) {
  const { org } = useOrg();
  return (
    <div className="note" style={{ marginTop: 6 }}>
      <b>{c.code}, {c.clause}</b> <span className="small">— {c.title}</span>
      <div className="small">{/^ред/i.test(c.edition) ? c.edition : `ред. ${c.edition}`} · {c.state_
label}{c.last_checked_at ? ` · проверено ${d(c.last_checked_at, org.timezone)}` : ''}{c.next_check_a
t ? ` · след. проверка ${d(c.next_check_at, org.timezone)}` : ''}</div>
      {c.quote ? <blockquote style={{ margin: '6px 0 0', paddingLeft: 10, borderLeft: '3px solid var
(--line)' }}>«{c.quote}»</blockquote> : <div className="small" style={{ marginTop: 4 }}>{c.text}</di
v>}
    </div>
  );
}

/** Ответ AI в формате Т3 v2 §11; предложенное действие становится задачей только по решению человек
а. */
export function AiAnswerView({ r }: { r: AiResult }) {
  const { org, can } = useOrg();
```

```
  const [decided, setDecided] = useState<Record<number, string>>({});
  const [role, setRole] = useState('haccp_lead');
  const [fb, setFb] = useState<string | null>(null);
  const { busy, error, run } = useSubmit();
  const a = r.answer;
  const decideItem = (i: number, decision: 'task_created' | 'rejected') => run(async () => {
    let note: string | null = null;
    if (decision === 'rejected') { note = prompt('Почему не подходит?'); if (!note) return; }
    await api(`/orgs/${org.id}/ai/jobs/${r.job_id}/decide`, { body: { itemIndex: i, decision, assign
eeRole: decision === 'task_created' ? role : undefined, note } });
    setDecided({ ...decided, [i]: decision === 'task_created' ? 'задача создана' : 'отклонено' });
  });
  return (
    <div className="card">
      <div className="between"><h2 style={{ margin: 0 }}>AI-помощник</h2>{a && <span className={`bad
ge ${CONF[a.confidence][1]}`}>уверенность: {CONF[a.confidence][0]}</span>}</div>
      <p className="small muted">AI объясняет и предлагает. Решения принимает ответственный; AI ниче
го не утверждает, не закрывает и не списывает.</p>
      {error && <div className="err">{error}</div>}
      {r.message && <div className={r.status === 'unavailable' ? 'note' : 'err'}>{r.message}</div>}
      {r.unsupported.length > 0 && <p className="small">Значения без источника: {r.unsupported.join(
', ')}.</p>}
      {a && (
        <>
          <h3>Что обнаружено</h3><p>{a.found}</p>
          <h3>Почему важно</h3><p>{a.why}</p>
          <h3>Что сделать сейчас</h3>
          {can('task.read') && a.do_now.length > 0 && (
            <div className="row small" style={{ marginBottom: 6 }}>Задачу назначить:
              <select aria-label="Исполнитель задачи" value={role} onChange={(e) => setRole(e.target
.value)} style={{ width: 'auto' }}>{ROLES.map(([k, v]) => <option key={k} value={k}>{v}</option>)}</
select></div>
          )}
          <ul className="list">{a.do_now.map((x, i) => (
            <li key={i} className="between"><span>{x}</span>
              {decided[i] ? <span className="badge">{decided[i]}</span> : (
                <span className="row" style={{ flexWrap: 'nowrap' }}>
                  <button className="btn small" disabled={busy} onClick={() => decideItem(i, 'task_c
reated')}>В задачу</button>
                  <button className="btn small secondary" disabled={busy} onClick={() => decideItem(
i, 'rejected')} aria-label={`Отклонить: ${x}`}>×</button>
                </span>
              )}</li>
          ))}</ul>
          {a.dont.length > 0 && <><h3>Нельзя до решения</h3><ul style={{ margin: 0, paddingLeft: 18
}}>{a.dont.map((x) => <li key={x}>{x}</li>)}</ul></>}
          <dl className="kv" style={{ marginTop: 10 }}><dt>Подтверждает</dt><dd>{a.who_confirms}</dd
><dt>Закроет проблему</dt><dd>{a.closes_when}</dd></dl>
          <h3>Источник</h3>
          {r.citations.length ? r.citations.map((c, i) => <CitationCard key={i} c={c} />) : <p class
```

```
Name="small muted">Ответ без ссылки на утверждённое требование — относитесь к нему как к подсказке,
а не к норме.</p>}
          {r.bad_citations.length > 0 && <p className="small">Отброшены недостоверные ссылки: {r.bad
_citations.map((b) => `${b.source} (${b.reason})`).join('; ')}.</p>}
          <h3>Ограничения</h3><p className="small">{a.limitations}</p>
          {fb ? <p className="small muted">{fb}</p> : (
            <div className="row small">Ответ помог?
              <button className="btn small secondary" disabled={busy} onClick={() => run(async () =>
 { await api(`/orgs/${org.id}/ai/jobs/${r.job_id}/decide`, { body: { decision: 'helpful' } }); setFb
('Спасибо.'); })}>Да</button>
              <button className="btn small secondary" disabled={busy} onClick={() => run(async () =>
 { await api(`/orgs/${org.id}/ai/jobs/${r.job_id}/decide`, { body: { decision: 'not_helpful' } }); s
etFb('Учтём.'); })}>Нет</button>
            </div>
          )}
        </>
      )}
      {!a && r.sources.length > 0 && (<><h3>Требования в реестре по теме</h3>{r.sources.map((c) => <
CitationCard key={c.id} c={c} />)}</>)}
      {r.model && <p className="small muted" style={{ marginTop: 10 }}>Модель: {r.model}. <Link href
={`/app/${org.id}/assistant/${r.job_id}`}>Запись в журнале →</Link></p>}
    </div>
  );
}

/** Кнопка «Объяснить с AI» для карточки отклонения или CAPA. */
export function AiPanel({ kind, entityId }: { kind: 'incidents' | 'capa'; entityId: string }) {
  const { org, can } = useOrg();
  const [r, setR] = useState<AiResult | null>(null);
  const { busy, error, run } = useSubmit();
  if (!can('ai.use')) return null;
  if (r) return <AiAnswerView r={r} />;
  return (
    <div className="card">
      {error && <div className="err">{error}</div>}
      <button className="btn block secondary" disabled={busy} onClick={() => run(async () => setR(aw
ait api<AiResult>(`/orgs/${org.id}/ai/${kind}/${entityId}`, { body: {} })))}>
        {busy ? 'AI думает…' : kind === 'incidents' ? 'Объяснить с AI и подсказать, что делать' : 'П
редложить причины и действия (AI)'}
      </button>
      <p className="small muted">Ответ — только по данным системы и утверждённым требованиям реестра
, со ссылкой на пункт и редакцию.</p>
    </div>
  );
}

// ===== src/components/AppShell.tsx =====
'use client';
import { useEffect, useState } from 'react';
import Link from 'next/link';
```

```
import { useRouter } from 'next/navigation';
import { Logo } from './Logo';
import { api } from '@/lib/api';

export interface Me { id: string; email: string; full_name: string; email_verified: boolean }

/**
 * Шапка как в прежнем ЦЕХ: ☰ (если есть меню разделов), логотип, справа имя и роль.
 * Нажатие на имя — выход из системы.
 */
export function TopBar({ me, roleLabel, onMenu, home = '/app' }: { me?: Me; roleLabel?: string; onMe
nu?: () => void; home?: string }) {
  const router = useRouter();
  return (
    <header className="topbar">
      {onMenu && <button className="hamburger" onClick={onMenu} aria-label="Меню разделов">☰</button
>}
      <Link href={home} className="logo" aria-label="ЦЕХ — на главную"><Logo height={30} eyes="var(-
-brand)" /></Link>
      {me && (
        <button className="who" title="Выйти" onClick={async () => { await api('/auth/logout', { bod
y: {} }); router.replace('/login'); }}>
          <span>{me.full_name}</span>
          <span className="role">{roleLabel ? `${roleLabel} · выйти` : 'выйти'}</span>
        </button>
      )}
    </header>
  );
}

/** Проверка сессии: без входа — на страницу входа с возвратом. bare — шапку рисует вызывающий (меню
 предприятия). */
export function AppShell({ children, bare = false }: { children: (me: Me) => React.ReactNode; bare?:
 boolean }) {
  const router = useRouter();
  const [me, setMe] = useState<Me | null | undefined>(undefined);
  useEffect(() => {
    api<{ user: Me | null }>('/auth/me').then((r) => {
      if (!r.user) router.replace(`/login?next=${encodeURIComponent(location.pathname)}`);
      else setMe(r.user);
    }).catch(() => setMe(null));
  }, [router]);
  const status = me === undefined ? <p className="muted">Загрузка…</p> : me === null ? <div classNam
e="err">Нет связи с сервером. Обновите страницу.</div> : null;
  if (bare && me) return <>{children(me)}</>;
  return (
    <>
      <TopBar me={me || undefined} />
      <main className="wrap">{status}{me && children(me)}</main>
    </>
```

```
  );
}

// ===== src/components/AuthShell.tsx =====
import { Logo } from './Logo';

/** Экран входа и регистрации в оформлении прежнего ЦЕХ: логотип на белой плашке, форма по центру. *
/
export function AuthShell({ title, children }: { title: string; children: React.ReactNode }) {
  return (
    <main className="narrow auth">
      <div className="brandblock"><div className="plate"><Logo height={88} eyes="#fff" animated /></
div></div>
      <div className="card">
        <h1>{title}</h1>
        {children}
      </div>
    </main>
  );
}

// ===== src/components/Evidence.tsx =====
'use client';
import { useCallback, useEffect, useRef, useState } from 'react';
import { useOrg } from '@/lib/org';
import { api } from '@/lib/api';
import { dt } from '@/lib/fmt';

interface File_ { id: string; filename: string; mime: string; size_bytes: number; sha256: string; no
te: string | null; uploaded_at: string; uploaded_by_name: string }

/** Доказательства к записи: фото с телефона или PDF; файл не удаляется и не заменяется — только доб
авляется. */
export function Evidence({ entityType, entityId, title = 'Доказательства' }: { entityType: string; e
ntityId: string; title?: string }) {
  const { org, can } = useOrg();
  const [files, setFiles] = useState<File_[] | null>(null);
  const [note, setNote] = useState('');
  const [busy, setBusy] = useState(false);
  const [err, setErr] = useState<string | null>(null);
  const input = useRef<HTMLInputElement>(null);
  const load = useCallback(() => api<File_[]>(`/orgs/${org.id}/evidence?entityType=${entityType}&ent
ityId=${entityId}`).then(setFiles).catch((e) => setErr(e.message)), [org.id, entityType, entityId]);
  const readable = can('evidence.read');
  useEffect(() => { if (readable) load(); }, [load, readable]);
  if (!readable) return null;

  async function upload(f: File) {
    setBusy(true); setErr(null);
    try {
```

```
      const fd = new FormData();
      fd.append('file', f); fd.append('entityType', entityType); fd.append('entityId', entityId); if
 (note) fd.append('note', note);
      const res = await fetch(`/api/v1/orgs/${org.id}/evidence`, { method: 'POST', body: fd, credent
ials: 'same-origin' });
      const j = await res.json().catch(() => null);
      if (!j?.ok) throw new Error(j?.error?.message || 'Не удалось загрузить файл.');
      setNote(''); await load();
    } catch (e) { setErr((e as Error).message); } finally { setBusy(false); if (input.current) input
.current.value = ''; }
  }

  return (
    <div className="card">
      <h2>{title}</h2>
      {err && <div className="err" role="alert">{err}</div>}
      {!files ? <p className="muted">Загрузка…</p> : files.length === 0 ? <p className="small muted"
>Файлов нет.</p> : (
        <ul className="list">{files.map((f) => (
          <li key={f.id}>
            <a href={`/api/v1/orgs/${org.id}/evidence/${f.id}`} target="_blank" rel="noopener">{f.fi
lename}</a>
            <div className="small muted">{dt(f.uploaded_at, org.timezone)} · {f.uploaded_by_name} ·
{Math.max(1, Math.round(f.size_bytes / 1024))} КБ{f.note ? ` · ${f.note}` : ''}</div>
            <div className="small muted" title="Контрольная сумма SHA-256">SHA-256: {f.sha256.slice(
0, 16)}…</div>
          </li>
        ))}</ul>
      )}
      {can('evidence.add') && (
        <>
          <label htmlFor={`en-${entityId}`}>Подпись к файлу (необязательно)</label>
          <input id={`en-${entityId}`} value={note} onChange={(e) => setNote(e.target.value)} placeh
older="Например: показания термометра" />
          <input ref={input} type="file" accept="image/jpeg,image/png,image/webp,image/heic,applicat
ion/pdf" aria-label="Файл доказательства" style={{ display: 'none' }}
            onChange={(e) => { const f = e.target.files?.[0]; if (f) upload(f); }} />
          <button type="button" className="btn block secondary" disabled={busy} onClick={() => input
.current?.click()}>{busy ? 'Загрузка…' : 'Добавить фото или PDF'}</button>
          <p className="small muted">До 10 МБ. Загруженный файл не удаляется и не заменяется.</p>
        </>
      )}
    </div>
  );
}

// ===== src/components/Logo.tsx =====
/**
 * Логотип «ЦЕХ»: башни с зубцами, «голова» с глазами и дуга — рисунок прежней системы ЦЕХ.
 * animated — как на экране загрузки прежней версии: маскот подпрыгивает и моргает, «Е» и «Х» въезжа
```

```
ют.
 */
export function Logo({ height = 32, eyes = 'var(--bg)', animated = false }: { height?: number; eyes?
: string; animated?: boolean }) {
  return (
    <svg viewBox="0 0 780 330" height={height} role="img" aria-label="ЦЕХ" className={animated ? 'ce
h-anim' : undefined} style={{ display: 'block', overflow: 'visible' }}>
      <path fill="currentColor" d="M44,48 L66,58 L84,46 L84,84 L90,84 L90,52 L99,44 L100,160 L180,16
0 L180,48 L203,58 L214,44 L214,82 L221,82 L221,74 L240,68 L246,80 L262,66 L262,268 L198,268 L198,246
 L44,246 Z" />
      <g className="ceh-mascot">
        <path fill="currentColor" d="M174,308 A58,50 0 0 1 290,308 L270,308 A38,32 0 0 0 194,308 Z"
/>
        <circle className="ceh-eye" cx="218" cy="238" r="8" fill={eyes} />
        <circle className="ceh-eye" cx="246" cy="238" r="8" fill={eyes} />
      </g>
      <path className="ceh-letter e" fill="currentColor" d="M314,66 L488,66 L488,118 L372,118 L372,1
32 L470,132 L470,178 L372,178 L372,194 L490,194 L490,246 L314,246 Z" />
      <path className="ceh-letter h" fill="currentColor" d="M506,66 L578,66 L616,124 L654,66 L724,66
 L652,154 L728,246 L656,246 L616,186 L576,246 L504,246 L580,154 Z" />
    </svg>
  );
}

// ===== src/components/OrgNav.tsx =====
'use client';
import { useEffect, useState } from 'react';
import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { useOrg } from '@/lib/org';
import { Logo } from './Logo';
import { TopBar } from './AppShell';

interface Item { href: string; label: string; show: boolean; match?: string[] }

/** Разделы предприятия по группам, как в боковом меню прежнего ЦЕХ; показываются только разделы рол
и. */
function useGroups(): [string, Item[]][] {
  const { org, can } = useOrg();
  const b = `/app/${org.id}`;
  const groups: [string, Item[]][] = [
    ['Оперативное', [
      { href: `${b}/shift`, label: 'Смена', show: can('measure.record') || can('receipt.record'), ma
tch: [`${b}/objects`, `${b}/report`] },
      { href: `${b}/problems`, label: 'Проблемы', show: can('incident.read'), match: [`${b}/incident
s`] },
      { href: `${b}/tasks`, label: 'Задачи', show: can('task.read') },
      { href: `${b}/stock`, label: 'Склад', show: can('lot.read'), match: [`${b}/lots`, `${b}/receip
ts`] },
      { href: `${b}/journals`, label: 'Журналы', show: can('journal.read') },
```

```
      { href: `${b}/pos`, label: 'Касса', show: can('pos.sell') },
      { href: `${b}/stop-list`, label: 'Стоп-лист', show: can('menu.read') },
    ]],
    ['ХАССП', [
      { href: `${b}/haccp`, label: 'План ХАССП', show: can('haccp.read') },
      { href: `${b}/capa`, label: 'CAPA', show: can('capa.read') },
      { href: `${b}/regulatory`, label: 'Нормативная база', show: can('reg.read') },
      { href: `${b}/assistant`, label: 'AI-помощник', show: can('ai.use') },
      { href: `${b}/pack`, label: 'Пакет для проверки', show: can('pack.export') },
    ]],
    ['Меню и продажи', [
      { href: `${b}/menu`, label: 'Блюда и ТТК', show: can('menu.read') },
      { href: `${b}/sales`, label: 'Продажи', show: can('sales.manage') },
      { href: `${b}/economics`, label: 'Экономика', show: can('econ.read') },
    ]],
    ['Управление', [
      { href: b, label: 'Управление', show: true },
      { href: `${b}/setup`, label: 'Объекты, товары, поставщики', show: can('control.manage') || can
('catalog.manage') },
      { href: `${b}/readiness`, label: 'Готовность к пилоту', show: can('pilot.read') },
      { href: `${b}/guide`, label: 'Памятки по ролям', show: true },
      { href: '/app', label: 'Мои предприятия', show: true },
    ]],
  ];
  return groups.map(([g, items]) => [g, items.filter((i) => i.show)] as [string, Item[]]).filter(([,
 items]) => items.length > 0);
}

/** Шапка с ☰ и выдвижное меню разделов (заменяет нижнюю панель, как в прежнем ЦЕХ). */
export function OrgChrome({ children }: { children: React.ReactNode }) {
  const { me, org, roleLabel } = useOrg();
  const groups = useGroups();
  const path = usePathname();
  const [open, setOpen] = useState(false);
  useEffect(() => { setOpen(false); }, [path]);
  useEffect(() => {
    if (!open) return;
    const esc = (e: KeyboardEvent) => { if (e.key === 'Escape') setOpen(false); };
    addEventListener('keydown', esc);
    return () => removeEventListener('keydown', esc);
  }, [open]);
  const b = `/app/${org.id}`;
  const active = (i: Item) => (i.href === b || i.href === '/app' ? path === i.href : [i.href, ...(i.
match || [])].some((h) => path.startsWith(h)));
  return (
    <>
      <TopBar me={me} roleLabel={roleLabel} onMenu={() => setOpen(true)} home={b} />
      <div className={`menu-overlay${open ? ' open' : ''}`} onClick={() => setOpen(false)} aria-hidd
en="true" />
      <nav className={`sidemenu${open ? ' open' : ''}`} aria-label="Разделы" aria-hidden={!open} ine
```

```
rt={!open}>
        <div className="menu-head"><Logo height={30} eyes="#fff" /><div className="small">{org.name}
</div></div>
        {groups.map(([g, items]) => (
          <div key={g}>
            <div className="menu-group">{g}</div>
            {items.map((i) => <Link key={i.href} href={i.href} className="menu-item" aria-current={a
ctive(i) ? 'page' : undefined}>{i.label}</Link>)}
          </div>
        ))}
      </nav>
      <main className="wrap">{children}</main>
    </>
  );
}

export function SitePicker() {
  const { sites, siteId, setSiteId } = useOrg();
  if (sites.length <= 1) return null;
  return (
    <select aria-label="Площадка" value={siteId || ''} onChange={(e) => setSiteId(e.target.value)} s
tyle={{ marginBottom: 12 }}>
      {sites.map((s) => <option key={s.id} value={s.id}>{s.name}</option>)}
    </select>
  );
}

// ===== src/components/useSubmit.ts =====
'use client';
import { useState } from 'react';

/** Состояние отправки формы: занято, ошибка, повторная отправка заблокирована на время запроса. */
export function useSubmit() {
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);
  async function run(fn: () => Promise<void>) {
    if (busy) return;
    setBusy(true); setError(null);
    try { await fn(); } catch (e) { setError((e as Error).message); } finally { setBusy(false); }
  }
  return { busy, error, setError, run };
}

// ===== src/lib/api.ts =====
'use client';

/** Вызов API v1 из браузера: { ok, data } или ошибка с понятным текстом. */
export class ApiError extends Error {
  constructor(public code: string, message: string, public status: number) { super(message); }
}
```

```
export async function api<T = unknown>(path: string, opts: { method?: string; body?: unknown } = {})
: Promise<T> {
  const res = await fetch(`/api/v1${path}`, {
    method: opts.method || (opts.body === undefined ? 'GET' : 'POST'),
    headers: opts.body === undefined ? undefined : { 'content-type': 'application/json' },
    body: opts.body === undefined ? undefined : JSON.stringify(opts.body),
    credentials: 'same-origin',
  });
  let json: { ok: boolean; data?: T; error?: { code: string; message: string } };
  try { json = await res.json(); } catch { throw new ApiError('NETWORK', 'Нет ответа от сервера. Про
верьте связь.', res.status); }
  if (!json.ok) throw new ApiError(json.error?.code || 'ERROR', json.error?.message || 'Ошибка.', re
s.status);
  return json.data as T;
}

// ===== src/lib/fmt.ts =====
/** Дата и время в часовом поясе предприятия; числа — с запятой. */
export function dt(ts: string | null | undefined, tz?: string, opts: Intl.DateTimeFormatOptions = {
day: '2-digit', month: '2-digit', hour: '2-digit', minute: '2-digit' }) {
  if (!ts) return '—';
  return new Date(ts).toLocaleString('ru-RU', { ...opts, timeZone: tz });
}
export function d(ts: string | null | undefined, tz?: string) {
  return dt(ts, tz, { day: '2-digit', month: '2-digit', year: 'numeric' });
}
export function n(v: string | number | null | undefined) {
  if (v === null || v === undefined || v === '') return '—';
  return String(Number(v)).replace('.', ',');
}
export function ago(ts: string | null | undefined) {
  if (!ts) return '';
  const min = Math.round((Date.now() - new Date(ts).getTime()) / 60e3);
  if (min < 1) return 'только что';
  if (min < 60) return `${min} мин назад`;
  const h = Math.round(min / 60);
  return h < 24 ? `${h} ч назад` : `${Math.round(h / 24)} дн назад`;
}

// ===== src/lib/op.ts =====
'use client';
import { useState } from 'react';

/**
 * Ключ операции для идемпотентных действий склада: один и тот же, пока действие не выполнено, —
 * повторное нажатие или повтор после обрыва связи не спишет второй раз. После успеха — новый ключ.
 */
export function useOperationId() {
  const make = () => (typeof crypto !== 'undefined' && 'randomUUID' in crypto ? crypto.randomUUID()
```

```
: `${Date.now()}-${Math.random().toString(36).slice(2)}`);
  const [id, setId] = useState(make);
  return { id, next: () => setId(make()) };
}

// ===== src/lib/org.tsx =====
'use client';
import { createContext, useCallback, useContext, useEffect, useState } from 'react';
import { api } from './api';
import type { Me } from '@/components/AppShell';

export interface Site { id: string; name: string; address: string | null; status: string }
export interface OrgCtx {
  me: Me;
  org: { id: string; name: string; status: string; timezone: string };
  role: string; roleLabel: string; permissions: string[]; siteIds: string[];
  sites: Site[]; siteId: string | null; setSiteId: (id: string) => void;
  can: (p: string) => boolean; reloadSites: () => Promise<void>;
}

const Ctx = createContext<OrgCtx | null>(null);

export function useOrg(): OrgCtx {
  const c = useContext(Ctx);
  if (!c) throw new Error('useOrg вне OrgProvider');
  return c;
}

/** Предприятие, права участника и площадки; выбранная площадка запоминается на устройстве. */
export function OrgProvider({ orgId, me, children }: { orgId: string; me: Me; children: React.ReactN
ode }) {
  const [state, setState] = useState<Omit<OrgCtx, 'setSiteId' | 'can' | 'reloadSites' | 'siteId' | '
me'> | null>(null);
  const [siteId, setSite] = useState<string | null>(null);
  const [err, setErr] = useState<string | null>(null);
  const key = `ceh.site.${orgId}`;

  const reloadSites = useCallback(async () => {
    const sites = await api<Site[]>(`/orgs/${orgId}/sites`);
    setState((s) => (s ? { ...s, sites } : s));
  }, [orgId]);

  useEffect(() => {
    (async () => {
      try {
        const r = await api<{ organization: OrgCtx['org']; membership: { role: string; role_label: s
tring; permissions: string[]; site_ids: string[] } }>(`/orgs/${orgId}`);
        const sites = (await api<Site[]>(`/orgs/${orgId}/sites`)).filter((s) => s.status === 'active
');
        setState({ org: r.organization, role: r.membership.role, roleLabel: r.membership.role_label,
```

```
 permissions: r.membership.permissions, siteIds: r.membership.site_ids, sites });
        let saved: string | null = null;
        try { saved = localStorage.getItem(key); } catch { /* приватный режим */ }
        setSite(sites.find((s) => s.id === saved)?.id || sites[0]?.id || null);
      } catch (e) { setErr((e as Error).message); }
    })();
  }, [orgId, key]);

  if (err) return <div className="err">{err}</div>;
  if (!state) return <p className="muted">Загрузка…</p>;
  const value: OrgCtx = {
    ...state, me, siteId,
    setSiteId: (id) => { setSite(id); try { localStorage.setItem(key, id); } catch { /* ignore */ }
},
    can: (p) => state.permissions.includes(p), reloadSites,
  };
  return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
}
```